Lightning Link

An Experimental Low-Latency UDP Networking Layer with Client-Side Prediction for a Deterministic 2-D Multiplayer Game

Shah Jahan Ishaq
University of Hong Kong
School of Computing & Data Science
3036095180

Supervisor: Dr. Chenshu Wu

April 19, 2026

Abstract

Real-time multiplayer games require that a player's input be reflected on screen quickly enough to feel causal, while the game's authoritative state remains consistent across geographically distributed participants, two requirements that the wide-area Internet, with its tens-to-hundreds of milliseconds of round-trip delay, jitter, and packet loss, makes mutually difficult to satisfy. This report describes Lightning Link, a C++20 prototype that implements and empirically evaluates the standard family of latency-hiding techniques used by production game networking stacks: an optimized path comprising a UDP transport, a compact binary wire protocol, a dedicated network receiver thread, client-side prediction with server reconciliation, and entity interpolation; and a deliberately naive baseline path comprising TCP, a text wire protocol, and no client-side latency hiding. Both paths are exercised through identical scenarios, manual two-player sessions and scripted bot harnesses, under a paired in-application network conditioner that imposes symmetric latency, jitter, and loss with shared pseudo-random seeds, and both emit per-input and time-series logs from which a Python analysis pipeline produces the report's figures. Across a five-condition main matrix, an ablation matrix that decomposes the bundled optimized path into its constituent techniques, a packet-loss sweep, and a participant-count sweep, the optimized path reduces median perceived input-to-visible delay from approximately 150ms to a single rendering frame at the 100ms round-trip operating point, with prediction accounting for the great majority of the improvement and interpolation visibly suppressing the snapshot-cadence stutter on remote characters under loss. The report contributes a reproducible, instrumented A/B testbed and a quantitative decomposition of where each optimization's contribution comes from.

Acknowledgements

I am truly grateful to my professors for the support and guidance they offered me throughout the course of this project. I extend heartfelt thanks to my supervisor for the project, **Professor Chenshu Wu**, whose expert advice, steady encouragement, and insightful feedback played a pivotal role in the development of this study's analytical approach and methodological strength.

I would also like to acknowledge Professor Locky Law whose instruction in the CAES Technical English for Computer Science course significantly improved my skills in communicating complex technical matters with clarity and precision

Finally, I wish to thank the wider academic community and open data resources, whose materials and perspectives formed the essential foundation for this research.

TABLE OF CONTENTS

CHAPTER 1. INTRODUCTION 5

1.1 Project background 5
1.2 Project objectives 6
1.3 Requirements 8
1.4 Contributions 11
1.5 Report structure 12

CHAPTER 2. PROJECT BACKGROUND AND LITERATURE REVIEW 14

2.1 Project background: the real-time multiplayer problem 14
2.2 What is available today 16
2.3 Shortcomings of what is available 18
2.4 Why undertake this project (motivation) 20
2.5 Gap this project addresses 22

CHAPTER 3. PROJECT METHODOLOGY 24

3.1 System architecture 24
3.2 Shared foundations 26
3.3 Wire protocols and message formats 29
3.4 The authoritative server 33
3.5 Client structure and rendering 37
3.6 Client-side latency hiding 41
3.7 Concurrency model 47
3.8 Instrumentation and observability 50
3.9 Build system and reproducibility 53
3.10 Evaluation pipeline 54

CHAPTER 4. EXPERIMENTS AND RESULTS 58

4.1. Experimental setup 58
4.2. Headline result: wire delay versus perceived delay 61
4.3. Decomposition: where does the gain come from? 63
4.4 Behaviour under degraded networks 65
4.5 Cost: bandwidth and snapshot stability 68
4.6 Scaling with participant count 70
4.7 Threats to validity 73

CHAPTER 5. CONCLUSION AND FUTURE WORK 74

5.1 Summary of findings 74
5.2 Limitations 74
5.3 Future work 75
5.4 Reflection on learning outcomes 75

CHAPTER 6. REFERENCES 77

APPENDICES 79

Chapter 1. Introduction

1.1 Project background

1.1.1 Real-time networked interactive systems as a domain

A growing class of software systems requires multiple geographically distributed participants to share a live, evolving state while each participant experiences the system as if it were local. Competitive multiplayer games are the clearest example, but the same pattern also appears in collaborative editors, telepresence systems, remote control of robots and vehicles, financial trading interfaces, and industrial digital twins.

These systems are harder to engineer than either offline applications or conventional request-response services. Offline applications have no network delay, and web services can often tolerate noticeable waiting times. Real-time interactive systems cannot: they must converge on a consistent shared state while also giving each user visible feedback within a few tens of milliseconds. Reconciling these two demands is the core problem addressed by this project.

1.1.2 The latency problem in competitive multiplayer games

Competitive multiplayer games expose this problem especially clearly. On a public network, the time between a player's input and the server's acknowledgement is often tens or hundreds of milliseconds, and that delay cannot be removed by application code because it is constrained by physical distance and routing topology.

Human perception makes this difficult. A delay of around 100ms is often enough for an action and its visible effect to feel separate, while much lower delays can still affect competitive feel. A naive client that waits for server confirmation before moving the local player will therefore feel unresponsive even on a good connection.

1.1.3 Why transport and client-side techniques matter

Over time, bandwidth has improved far more than latency. As a result, the main challenge is no longer simply how many bytes can be sent, but how communication can be structured so that fixed network delay becomes less visible to the user.

This cannot be solved entirely by the transport layer. The application itself must decide what can be lost, what can be predicted locally, and what must be reconciled later. In practice, this leads to two groups of techniques: transport and protocol techniques such as UDP and compact binary messages, and client-side techniques such as prediction, interpolation, and reconciliation.

The central hypothesis of this project is that actual network round-trip time can remain essentially unchanged while the user's perceived input delay becomes much smaller. The rest of this report tests that claim experimentally.

1.2 Project objectives

The project turns that hypothesis into four concrete objectives. Together, these objectives define the scope of the work.

1.2.1 Construct a playable client-server interactive system from scratch

The first objective is to build a complete client-server interactive system in modern C++20 from the socket layer upward, without relying on existing game-networking middleware or an engine. The game world is intentionally simple: a two-dimensional top-down arena in which several players can move freely, because the focus is on networking and simulation rather than gameplay complexity.

1.2.2 Implement a deliberately weak baseline transport and protocol

The second objective is to build a fully working baseline version of the same game that is intentionally naive. It uses TCP, a verbose textual wire format, and no client-side latency hiding: local movement happens only after server acknowledgement, and remote players are shown with no smoothing.

This baseline is included as a control for comparison. Without it, it would be difficult to attribute any measured improvement to a specific optimization.

1.2.3 Implement an optimized transport, protocol, and client simulation

The third objective is to build an optimized version of the same game using UDP, a compact binary wire format, and client-side latency-hiding techniques. The optimized client applies its own inputs immediately, then reconciles against authoritative snapshots when they arrive, and it renders other players with interpolation to smooth out network jitter.

These optimizations can be enabled or disabled independently so that their individual contributions can be measured.

1.2.4 Build a reproducible, scripted evaluation pipeline

The fourth objective is to make the project reproducible. The system includes an automated bot harness, scripted experiment runners, and a Python analysis pipeline that processes logs and generates the figures used later in the report.

This matters because the project's contribution is partly quantitative. A result that cannot be reproduced from the repository alone is weaker as evidence, so reproducibility is treated as a core objective rather than an implementation detail.

1.3 Requirements

The objectives above are translated into testable requirements. These are divided into functional requirements, non-functional requirements, and scope boundaries so the report states clearly what the system must do, what quality properties it must satisfy, and what is outside the project's scope.

1.3.1 Functional requirements

FR-ID	Requirement
FR-1	A central server process accepts client connections, maintains an authoritative two-dimensional simulation, and broadcasts state updates to all connected clients.
FR-2	A client process renders the simulation in a window, samples player input from the keyboard or from a scripted pattern and transmits inputs to the server.
FR-3	At least two and at most eight clients may participate in a single session simultaneously.
FR-4	A client may join an in-progress session, observe other players, move freely within the arena bounds, and leave cleanly without disturbing the remaining clients.
FR-5	Two parallel implementations of the system exist: an optimized implementation using UDP and a compact binary wire format, and a baseline implementation using TCP and a verbose textual wire format. The user-visible game logic is identical in both.
FR-6	The optimized client applies its own inputs immediately and reconciles against the server's authoritative snapshots when they arrive. The baseline client renders only authoritative state.

FR-7	The optimized client renders other players using time-delayed interpolation between successive authoritative snapshots. The baseline client renders other players at the latest received state with no interpolation.
FR-8	The optimized client exposes runtime toggles for prediction and interpolation, and command-line switches that fix those toggles at startup.
FR-9	The server detects clients that have not produced input within a bounded interval and removes them from the simulation.
FR-10	Each client process emits structured per-input and periodic-aggregate logs in CSV form for the entire session.
FR-11	A scripted experimental harness drives the system through controlled experimental matrices and produces the figures presented later in the report.
FR-12	Both transports support an in-application network conditioner parameterized by added latency, jitter, and packet-loss probability.

Table 1.1: Functional Requirements of the project

1.3.2 Non-functional requirements

NFR-ID	Requirement
NFR-1	Determinism. For a fixed input sequence and fixed pseudo-random seed, repeated runs produce statistically identical aggregate results.
NFR-2	Reproducibility. Every figure in Chapter 4 can be regenerated from the repository with one documented command.
NFR-3	Measurability. The implementation separately instruments input-to-acknowledgement and input-to-visible-effect intervals.

NFR-4	Fairness of comparison. Features added to one mode must either be added to the other or recorded as deliberate deviations.
NFR-5	Portability. The codebase builds and runs on macOS and Linux on Apple Silicon and x86-64 using only C++20, CMake, and SFML 3.0.
NFR-6	Robustness. The decoder rejects truncated and over-length packets safely, and a self-test checks round-trip correctness plus malformed input handling.
NFR-7	Operational simplicity. No kernel modules, elevated privileges, or external traffic-shaping tools are required.
NFR-8	Honesty. Any deliberate departure from the written specification is recorded in a deviations log.

Table 1.2: Non-functional requirements of the project

1.3.3 Scope boundaries

OS-ID	Boundary
OS-1	The game world is a two-dimensional top-down arena with simple kinematic motion. No physics, collision, or 3D rendering is in scope.
OS-2	The maximum supported session size is eight concurrent clients. Larger-scale sharding and area-of-interest culling are out of scope.
OS-3	All experiments run on a single host using loopback, with impairments injected by the in-application conditioner. Real wide-area measurements are not in scope.
OS-4	The project provides no security guarantees: no encryption, authentication, authorization, or anti-cheat.
OS-5	No NAT traversal, relay infrastructure, or peer-to-peer fallback path is provided.

OS-6	The protocol uses full state snapshots and does not investigate delta compression or area-of-interest filtering.
OS-7	The server does not perform retroactive lag compensation.

Table 1.3: Scope boundaries of the project. Defines what is outside scope (OS)

1.4 Contributions

The project does not claim that prediction, interpolation, ack-based reconciliation, or compact binary framing are new techniques. Its contribution is that these techniques are integrated into a single small readable codebase, paired with a weak baseline, and measured using a reproducible experimental pipeline.

The project contributes five things: a complete two-mode interactive system, an in-application network conditioner, a scripted bot harness, a four-matrix experimental pipeline with analysis tooling, and quantitative evidence showing how much each technique contributes to perceived responsiveness.

1.4.1 A complete two-mode interactive system in a single repository

The repository contains a client and server written in modern C++20 that implement two parallel versions of the same game, one optimized and one deliberately weakened, while sharing one authoritative simulation. It includes the transports, wire formats, prediction and reconciliation logic, interpolation buffer, rendering and HUD layers, bot harness, and build system.

1.4.2 An in-application network conditioner that runs without privileges

The project includes a user-space network conditioner that can add latency, jitter, and packet loss without kernel-level traffic shaping. It is implemented consistently across both modes and is seeded so experiments remain deterministic and reproducible.

1.4.3 A scripted bot harness for headless, automated experiments

The client can run as a deterministic bot using circular, sinusoidal, zigzag, or seeded-random input patterns, with or without a window. This makes the experiments automated and practical even at the maximum supported player count.

1.4.4 A four-matrix experimental pipeline and a single-pass analyzer

The evaluation framework includes four orchestrated experiment matrices: optimized versus baseline, per-feature ablation, packet-loss sweep, and player-count scaling. A Python analyzer aggregates logs and produces the figures for Chapter 4 with confidence intervals.

1.4.5 A quantitative decomposition of perceived-delay reduction

The empirical contribution is a measured breakdown of how much improvement comes from each technique. The results show that wire-level acknowledgement latency is similar between modes, while user-perceived delay is much lower in the optimized stack, largely because of client-side prediction.

1.5 Report structure

Chapter 2 reviews the domain, prior work, limitations of existing solutions, and the gap this project addresses. Chapter 3 explains the implementation and the evaluation pipeline, Chapter 4 presents the experiments and results, and Chapter 5 discusses the findings, limitations, and future work.

Chapter 2. Project Background and Literature Review

2.1 Project background: the real-time multiplayer problem

2.1.1 Domain and scale

The class of system addressed by this project is broader than the games on which the prototype is modelled. Across contemporary software, several deployment classes share the same defining property: a piece of state evolves continuously and must be observed by many geographically distributed participants in something close to real time, while each participant simultaneously contributes inputs that mutate that state. Competitive online games are the most visible instance, with hundreds of millions of daily participants across globally distributed server regions. Real-time collaborative document editors, video conferencing tools, tele-operation of robots and vehicles, and industrial digital twins exhibit the same structural pattern with relaxed or tightened timing budgets respectively. The applications differ in their stakes and in the fidelity of state they share, but the underlying problem is the same: a distributed simulation must remain consistent across all participants on a network that is unreliable, variable in delay, and shared with other traffic, while feeling instantaneous to each participant individually.

Round-trip latency on a public wide-area network is bounded below by physical fiber propagation and above by the depth of the routing graph between the participant and the server [1]. Typical figures for residential connections are approximately 30ms within a metropolitan area, 60-100ms between cities on the same continent, and 150-250 ms between continents. These ranges have been remarkably stable for over a decade: the limiting term is the speed of light, not the engineering of the network.

2.1.2 What "responsive" means quantitatively

Three perceptual thresholds, drawn from the human-computer interaction literature and from later work specific to interactive games, give "responsive" a numerical meaning.

The first is the threshold of *causal coupling*: the longest delay between an input action and its visible consequence at which a user still experiences the two as a single causally linked event. The classical figure, established by Miller [2] and later restated by Card, Moran and Newell [3], is approximately 100ms. Beyond this threshold, the action and its consequence begin to feel like two distinct events and the system feels sluggish even if functionally correct.

The second is the threshold of *competitive parity*: the largest latency disadvantage a player can tolerate before their performance in adversarial real-time interaction begins to suffer measurably. Empirical studies of competitive multiplayer titles place this threshold in the range of 20-50 ms for fast-paced first-person genres, with more permissive figures for strategy and turn-based formats. The most-cited measurement is by Beigbeder et al. on *Unreal Tournament 2003* [4]; Claypool and Claypool subsequently surveyed the cross-genre evidence [5], and Sheldon et al. produced parallel measurements for *Warcraft III* showing the corresponding (much larger) tolerance for strategy games [6].

The third is the threshold of *imperceptibility*: the latency below which an additional millisecond of delay can no longer be detected, which most psychophysical studies place at single-digit milliseconds for visual feedback to manual input.

A naive client-server implementation, in which the local participant's avatar moves only after the server has acknowledged the input, sits well outside even the first threshold in any realistic deployment. At a 100 ms inter-city round trip the acknowledgement arrives at the boundary of causal coupling, and any added jitter pushes the felt response over that boundary on a non-trivial fraction of frames. At a 200 ms intercontinental round trip the implementation simply does not feel like a real-time system. This is the gap the techniques surveyed in §2.5 are designed to close.

2.1.3 The anatomy of a networked game loop

It helps to fix the canonical decomposition of a real-time networked interactive system into the stages each input passes through before its consequence reaches the screen. The same five stages appear, with cosmetic vocabulary differences, in every published account of how a networked game loop is organized, from the Quake III networking model [7] through Bernier's account of the Source engine's protocol [8] and Valve's developer documentation [9].

The five stages are *input capture* on the client, *transmission* of the input to the authoritative server, *authoritative simulation* on the server, *propagation* of the new authoritative state back to the clients and *rendering* on the client. In a naive implementation, the local participant's input is felt only after all five stages have completed, because the implementation refuses to advance the local view until the authoritative state has confirmed it. The total felt latency is therefore the sum of all five stages, dominated by the two network traversals. Any technique that hopes to reduce felt latency without changing the network must intervene at one of the stages that does not require a network round trip: by short-circuiting the local view of the participant's own state ahead of authoritative confirmation, by smoothing the perceived arrival of authoritative state over time, or by changing the representation in which state is propagated so less time is spent on the wire. The remainder of this chapter examines what existing systems, and the published literature provide along each of these axes, what they fail to provide, and what specific subset of techniques the prototype of chapters 3 and 4 implements.

2.2 What is available today

A developer who sets out to build a real-time networked interactive system today is not starting from a blank slate. Several distinct categories of pre-existing artefact are available off the shelf, each addressing a different layer of the problem. This section surveys those categories at the level of detail required to support the comparative discussion in the remainder of the chapter; a more focused review of the specific techniques each artefact embodies is deferred to section 2.5.

2.2.1 Commercial game engines with built-in networking

The first and most heavily used category consists of full-stack engines that ship networking as a built-in subsystem. Unreal Engine [10] provides a replication model based on networked actors and remote procedure calls organized around a server-authoritative topology and is the de facto standard for first- and third-person multiplayer titles. Unity ships *Netcode for GameObjects* as a first-party solution [11] and is widely used in conjunction with the open-

source *Mirror* and *FishNet* libraries [12], [13] which provide similar abstractions. Valve's Source and Source 2 engines, which power *Counter-Strike*, *Team Fortress 2*, and *Dota 2*, use a snapshot-based model in which the server periodically sends each client a delta-compressed description of the world relative to a previously acknowledged baseline [9]. All three families implement, in some form, the same core latency-hiding techniques surveyed in §2.5: authoritative simulation, client-side prediction, entity interpolation, and lag compensation for hit-scan interactions [8].

2.2.2 Open-source networking libraries

A second category consists of libraries that provide the transport and reliability layer alone, without imposing a particular game architecture above it. *ENet* [14] is a long-standing C library that layers ordered, reliable, and sequenced channels on top of UDP. Valve's *GameNetworkingSockets* [15] exposes a more recent, encrypted, production-grade transport with selective reliability and built-in network conditioning. *yojimbo* and *netcode.io* by Glenn Fiedler [16], [17] focus on secure connection establishment over UDP with client authentication and replay protection. A more recent strand of work uses QUIC, originally designed for HTTP delivery and standardised as RFC 9000 [18], as a substrate for game-style traffic, since its combination of reliable streams and unreliable datagrams over a single encrypted channel is increasingly attractive for new projects. All of these libraries solve the transport problem; none implement prediction, interpolation, or any other technique that operates inside the application's simulation loop.

2.2.3 Reference implementations and educational accounts

A third category consists of implementations and written accounts explicitly intended to be read and learned from. The source release of *Quake III Arena* in 2005 [7] made the first mainstream commercial implementation of client-side prediction and snapshot-based state propagation available for inspection. Bernier's 2001 GDC paper [8] gives a precise account of the techniques used in the Source engine and remains the canonical reference for the modern formulation of lag compensation. Gambetta's tutorial series *Fast-Paced Multiplayer* [19] presents the same set of techniques in a deliberately minimal browser-based prototype, and Fiedler's long-running *Gaffer on Games* article series [20], much of which subsequently informed the design

of *yojimbo* and *netcode.io*, covers prediction, interpolation, snapshot compression, and reliability over UDP at a similar level of accessibility.

2.2.4 Adjacent transport-level infrastructure

A fourth category is broader and is mentioned for completeness. Modern operating systems and standard libraries ship secure transport primitives, DTLS for datagram transport security and the QUIC protocol referenced above, which encrypted game-networking libraries increasingly build upon. Browser-resident applications have access to WebRTC data channels [21], which provide a UDP-like substrate suitable for real-time peer-to-peer or relayed traffic. For empirical evaluation, kernel-level traffic shaping is available on Linux through tc netem, on macOS through pfctl and dummynet, and on Windows through the third-party Clumsy tool; each allows controlled latency, jitter, and loss to be introduced at the operating-system layer without modifying the application.

Section 2.3 examines, in turn, the specific ways in which each of these categories is unsuitable for the particular task this project sets itself.

2.3 Shortcomings of what is available

2.3.1 Commercial engines are opaque for measurement

Unreal, Unity, and Source ship complete networking stacks in which prediction, interpolation, and snapshot delta-compression are bundled together and enabled by default. None of the three exposes a supported way to disable a single technique while leaving the others intact, and the source code of the relevant subsystems is either closed (Unity Netcode, Source 2) or large enough that an attribution experiment would itself be a final-year project. Bernier's account of Source [8] and Valve's developer wiki [9] explain *what* the engine does, but neither permits the kind of A/B comparison required to answer the question "how much does prediction alone reduce perceived delay". The same engines are also unsuitable as a teaching artefact: a reader who wishes to

understand the implementation of client-side prediction would have to navigate hundreds of thousands of lines of supporting infrastructure to find it.

2.3.2 Open-source libraries solve transport, not the game loop

The libraries surveyed in §2.2.2, ENet [14], GameNetworkingSockets [15], yojimbo and netcode.io [16], [17], and the QUIC ecosystem [18], are highly engineered solutions to the problem of moving bytes reliably or unreliably over UDP. None of them implements client-side prediction, server reconciliation, or entity interpolation, because those techniques live above the transport in the application's simulation loop. A developer who adopts one of these libraries has solved the easier half of the problem and inherited the harder half unchanged. Their documentation is API-focused rather than pedagogical, and none of them ships a deliberately naive baseline transport against which their own behaviour can be compared in an A/B fashion.

2.3.3 Reference implementations are descriptive, not measured

The educational accounts in §2.2.3, Quake III's source release [7], Bernier's Source paper [8], Gambetta's tutorial series [19], and Fiedler's article archive [20], explain the techniques clearly and at the right level of detail for a serious reader. What they do not do is *quantify* them under controlled conditions against a baseline: none of these references reports the perceived delay reduction attributable to prediction in isolation, the bandwidth difference attributable to a binary wire format in isolation, or the jitter absorbed by interpolation in isolation, in numbers traceable to a published harness. Most of the accompanying example code, where it exists, is browser-based JavaScript [19] rather than a small C++ artefact a systems student can read end-to-end.

2.3.4 Academic evaluations are fragmented

The empirical literature complements rather than replaces the references above. Beigbeder et al. [4], Claypool and Claypool [5], and Sheldon et al. [6] each isolate one variable, added latency, in their case, and quantify its effect on player performance, but their measurements address an

upstream question (how much latency hurts) rather than the downstream one (how much each individual technique recovers). Studies of deployed protocols, including academic measurement work on QUIC [22] and on production game telemetry [23], typically report behaviour at one operating point rather than across systematic matrices, and few publish their harness or raw data in a form a third party can re-run. The survey by Suznjevic and Matijasevic [24] makes this gap explicit: networked-game evaluation is methodologically uneven, and reproducible per-technique decompositions are rare.

These four shortcomings, taken together, define the gap §2.5 returns to: there is no compact, readable, reproducible artefact that implements both a deliberately naive baseline and an optimized path side by side, exposes per-technique ablation, and ships the harness and raw data alongside the code.

2.4 Why undertake this project (motivation)

The shortcomings catalogued in section 2.3 explain why a small, self-contained, instrumented artefact is the right vehicle for the question this project asks. They do not, by themselves, explain why this particular project was undertaken by this particular author. That explanation has four distinct strands, of which the first is personal and the remaining three are technical.

2.4.1 Personal learning

The dominant motivation is personal. The undergraduate computer-systems curriculum covers networking, operating systems, concurrency, and systems programming as separate courses, each typically exercised through small, isolated assignments. A real-time multiplayer prototype is one of the few self-contained projects that exercises all of them simultaneously and at production-relevant intensity. The networking content of the curriculum, normally encountered as textbook descriptions of the OSI layers and the TCP state machine [25], becomes concrete only when the student is required to choose between TCP and UDP for a specific workload, design a wire format, and reason about round-trip time, jitter, and packet loss as runtime variables rather than

examination topics. The same is true of operating-systems content: blocking versus non-blocking I/O, select with timeouts, signal-driven shutdown, and kernel scheduling of a multi-threaded process are encountered as functioning code rather than as diagrams. Concurrency primitives, std::mutex, std::condition_variable, std::atomic, and the producer-consumer queue pattern [26], are likewise easier to internalise after debugging the class of bugs that only manifests under contention. Finally, the project exercises modern C++ ownership, RAII, and move semantics [27] across a multi-module codebase larger than any single course assignment, together with the supporting build-system and cross-platform tooling decisions that any non-trivial systems project requires.

A secondary personal motivation is experimental: the project is an opportunity to practise the empirical-evaluation discipline that the literature in §2.3 was identified as lacking. Designing a reproducible harness, choosing metrics that defend a specific claim, replicating runs, reporting confidence intervals, and resisting the temptation to cherry-pick favourable results are all skills that an examination-based curriculum has limited opportunity to teach.

2.4.2 Pedagogical motivation

A self-contained C++20 codebase, small enough that a reader can in principle understand every line, is a more useful teaching artefact for a student who wishes to study client-side prediction or interpolation than any production game engine. The prototype implements the techniques surveyed in §2.5 from the socket upwards rather than as configuration on top of a pre-existing framework, exposing where each millisecond of perceived delay originates and where it is removed. The deliberately naive baseline is preserved alongside the optimized path so that the two can be diffed literally rather than reasoned about hypothetically.

2.4.3 Methodological motivation

§2.3 identified the descriptive style of the existing literature as one of its main shortcomings: claims of the form "prediction helps" are common, but quantitative claims of the form "prediction reduces perceived delay from 130 ms to single-digit ms at a 100 ms round trip" are not. The

presence of compile-time and command-line ablation surfaces in this prototype, combined with the four scripted experiment matrices described in chapter 3, allows each technique to be evaluated in isolation and reported with the relevant confidence intervals. This is intended as a small contribution to the per-technique decomposition gap identified in §2.3.4.

2.4.4 Practical motivation

The techniques studied here are not specific to games. Any soft-real-time distributed interactive system, remote tele-operation, shared editing, virtual-reality co-presence, industrial digital twins, faces the same trade-off between authoritative consistency and felt responsiveness, and applies broadly the same families of solution. A reproducible baseline-versus-optimized harness of the kind built here therefore has reuse value beyond its original domain, both as a sandbox for testing further protocol ideas and as a checked example a future student can extend.

2.5 Gap this project addresses

The shortcomings catalogued in §2.3 are not, individually, novel observations: production engines have always been opaque, transport libraries have always lived below the simulation loop, and educational accounts have always been descriptive rather than measured. What §2.3 establishes is that no single existing artefact addresses all four shortcomings simultaneously. The contribution this project sets itself is to fill that gap at a deliberately small scale: a single C++20 codebase, small enough that a serious reader can read every line, that implements *both* a deliberately-naive baseline transport (TCP, line-based text) and an optimized path (UDP, binary, client-side prediction, entity interpolation) so that the two can be diffed line-by-line rather than reasoned about hypothetically.

To that core artefact, the project adds three further elements that the references in §2.2 do not jointly provide. First, per-technique ablation is built into the client itself: prediction and interpolation can each be disabled at start-up, allowing four mode variants between fully optimized and fully naive to be exercised against the same server. Second, four scripted experiment matrices, covering the main A/B comparison, per-technique isolation, a packet-loss sweep, and a player-

count scaling sweep, drive the system through automated bot clients and produce report-ready figures with a single command. Third, the raw measurement data and the analysis scripts are committed alongside the prototype, so that any quantitative claim made in chapter 4 is reproducible from the artefact and contestable by any subsequent reader.

The combination is modest. None of the four elements is individually novel; the contribution lies in their being addressed together, in one place, at a scale a reader can actually read. Chapters 3 and 4 describe the artefact and the measurements, respectively, in the detail required to substantiate that claim.

Chapter 3. Project Methodology

This chapter describes how the prototype is constructed. It is organised around the journey of an input through the system rather than around the directory layout of the source. Section 3.1 fixes the architectural vocabulary used throughout the chapter; sections 3.2 and 3.3 describe the shared substrate and the wire protocols that sit on top of it; sections 3.4 to 3.6 describe the authoritative server, the client's structural plumbing, and the client-side latency-hiding techniques that constitute the report's intellectual centre; section 3.7 gives a unified view of the concurrency model now that both sides have been described; sections 3.8 to 3.10 cover the instrumentation that produces the raw data analysed in chapter 4, the build configuration, and the experimental pipeline. Together the chapter discharges functional requirements FR-1 through FR-12 by inspection of the implementation; the non-functional requirements NFR-1 through NFR-8 are demonstrated in part here and in part by the experimental evidence in chapter 4.

3.1 System architecture

3.1.1 Two-mode design and shared code

The defining structural decision is that two parallel implementations of the same game share a single codebase, build, and authoritative simulation, but differ in their transport, wire format, and client-side latency-hiding techniques. The mode is fixed at process startup, because each owns a distinct socket and serialisation path. The optimized mode carries UDP datagrams encoded in a compact binary format (§3.3.1); the baseline opens a TCP byte stream and exchanges newline-delimited textual messages (§3.3.2).

Co-locating both modes in one repository is the mechanism by which the chapter 4 comparison is kept fair. The simulation itself, the constants that govern it, and the in-application network conditioner are all shared, the conditioner being mirrored at line-level granularity inside the baseline. Any feature added to one mode but omitted from the other is recorded in the deviations

log, in compliance with NFR-8. The deliberately weak baseline serves as the experimental control against which the optimized mode is measured (FR-5).

3.1.2 Process topology

The runtime topology is intentionally simple. A single server process listens on a well-known UDP port and a well-known TCP port and owns the authoritative copy of the simulation. Between two and eight client processes connect to it as independent OS processes; clients may join in progress, leave cleanly via an explicit disconnect, or be reaped by the server's liveness timeout (§3.4.2). The server has no client of its own and renders nothing.

Each client runs in one of two modes that are independent of the server's optimized-or-baseline choice: an interactive mode that opens a window and samples the keyboard, or a headless bot mode that generates a deterministic input pattern from one of four families (circular, sinusoidal, zigzag, seeded random) and may suppress window creation. The bot mode drives all of chapter 4's experiments, since it is the only practical means of running the eight-client stress condition reproducibly.

The system is reported here in a single-host configuration over the loopback interface, with impairments injected by the in-application conditioner (§3.2.3). Wide-area deployment is technically possible but lies outside the scope of these experiments (OS-3).

3.1.3 Module map

The codebase has three top-level source directories plus a build configuration and an evaluation toolchain.

The first contains code linked into both server and client: the cross-platform socket layer, the in-application network conditioner, the binary wire-format encoder and decoder, the CSV logging utilities, and the shared constants and types. Its contents are described in §3.2; the wire protocols built on top of them are the subject of §3.3.

The second hosts the server: the authoritative fixed-timestep simulation, the session manager, the snapshot broadcaster, and the parallel TCP implementation in baseline mode. Described in §3.4.

The third is the largest and is treated in two parts. Its structural contents, entry point, input sampler, bot harness, renderer, and the parallel baseline client, are in §3.5. Its latency-hiding contents, the network receiver thread, the snapshot queue, the client-side predictor and reconciler, the entity interpolator, and the ablation surface, are the subject of §3.6 and constitute the technical centre of the report.

A CMake build configuration produces three artefacts (server, client, and a self-test binary), described in §3.9; shell scripts and a Python analyzer implement the experimental pipeline (§3.10).

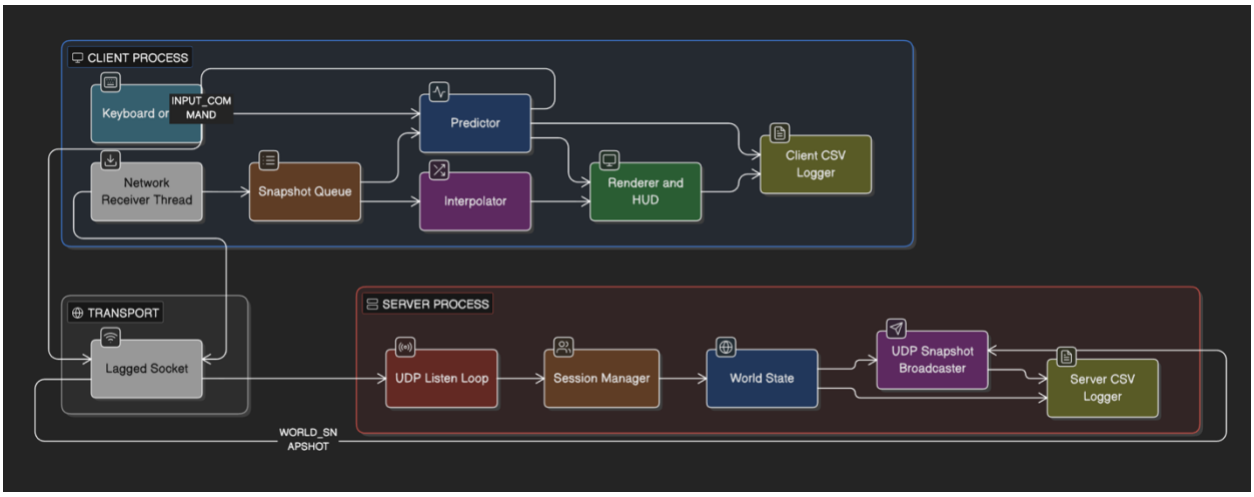

Figure 3.1: Component overview and data flow. The optimized path is illustrated; the baseline path has the same topological shape with TCP byte streams in place of UDP datagrams and with the predictor and interpolator removed.

3.2 Shared foundations

The shared substrate is the small set of facilities that both server and client depend on, and that must be implemented exactly once for the chapter-4 comparison to remain fair: the constants and primitive types that fix the simulation's parameters, a thin compatibility layer over the BSD socket

interface, the in-application network conditioner that injects controlled latency, jitter, and loss in place of kernel traffic shaping, the CSV logging utility, and a self-test target that exercises the wire format in isolation.

3.2.1 Simulation constants and shared types

A single shared header fixes every simulation parameter and is included by both processes, eliminating any possibility of disagreement on tick rate, arena dimension, or buffer size (NFR-1).

The simulation ticks at 60 Hz, the same rate at which each client samples its input. Authoritative state is broadcast at 20 Hz (every third tick), enough temporal resolution for smooth motion under interpolation while keeping per-client bandwidth low. The arena is 1280 × 720 units, players move at 240 units/s in unit-vector directions, the player representation is a circle of radius 18 units, and sessions hold two to eight participants. The header also fixes the latency-hiding parameters of §3.6 (120-entry input history, 32-entry snapshot history, 100 ms interpolation buffer), the §3.4.2 session parameters (3 s client timeout, 2 s entity staleness), and the 1 s log flush.

Three primitive types appear throughout: a 2D float vector; a player-state record carrying a 16-bit player id, position, velocity, and the sequence number of the most recent input the server has integrated for that player, the field on which all reconciliation rests; and the input command, which carries id, sequence number, and two single-byte signed coordinates constrained to {-1, 0, +1}.

3.2.2 Cross-platform socket compatibility layer

The system targets desktop POSIX environments (macOS and Linux) and uses the BSD socket interface directly. The compatibility layer collects the small handful of places where the two operating systems differ, most prominently the macOS requirement that an outbound socket address be tagged with its own length and exposes a uniform interface to the rest of the codebase, which is free of conditional compilation. It also defines a normalised endpoint type (IPv4 address plus port in network byte order, equality-comparable and hashable) used as the session-manager key in §3.4.2.

3.2.3 In-application network conditioner

The substrate's largest component is an in-application network conditioner that wraps a UDP socket and, on every send and receive, applies a configured one-way latency, uniform jitter, and Bernoulli loss probability. It is the mechanism by which the chapter 4 experiments simulate adverse network conditions on the loopback interface, where the bare network adds essentially no measurable delay or loss.

The choice to implement conditioning inside the application, rather than depend on kernel traffic-shaping tools, is deliberate. The two operating systems differ in their traffic-shaping interfaces, privilege requirements, and default availability; an in-application conditioner has none of these dependencies, runs in user space, and is fully determined by parameters supplied at startup. It is driven by a single seeded pseudo-random generator, so that two runs of the same experiment produce, modulo OS scheduling jitter, byte-for-byte identical wire traffic, the property on which NFR-1 (determinism) and NFR-2 (reproducibility) rest.

Four numerical parameters configure it: an outbound and an inbound latency (a single user-facing one-way value is split symmetrically), the half-width of a uniform jitter distribution, and a per-packet loss probability. Internally it maintains a priority queue of pending packets ordered by scheduled delivery time. On send, it consults the loss probability and either drops the packet or schedules it. On receive, it drains kernel-delivered packets, applies the inbound loss roll, and dequeues only those whose deadlines have matured. A periodic pump method, called at the tick rate, ensures queued outbound packets continue to be delivered during quiet periods.

The same conditioner is used by both endpoints on the optimized path. The baseline applies the same discipline at line granularity, sharing parameters and seed but operating on a different unit by necessity of its transport.

3.2.4 CSV logging

The shared substrate provides only the small thread-safe sink that instrumentation calls into; the metrics themselves are the subject of §3.8. The sink takes a file path and a header line at

construction, opens the file (writing the header on first creation, appending otherwise), serialises concurrent writers under a mutex, and accepts pre-formatted lines for append. A paired helper reports wall-clock time in milliseconds since the OS epoch, preferred to monotonic time because it is comparable across processes that share a host. A flush is requested explicitly at the end of each periodic log cadence (default 1 s), so a process killed mid-experiment leaves on disk every sample recorded up to the most recent cadence boundary.

3.2.5 Self-test target as a wire-format correctness contract

The build also produces a self-test executable whose only purpose is to exercise the encode and decode paths of every message type in §3.3 in isolation. It verifies five properties: byte-exact round-trip equality of every field; encoder output length matching the analytical form of §3.3; type-tag peek-ability without consuming the message; encoder refusal (writing nothing, returning length zero) when the destination buffer is too small; and decoder failure when the source buffer has been truncated by even a single byte. The latter two properties jointly discharge the robustness requirement NFR-6 and are checked under a macro that aborts even with assertions disabled, so a regression in the bounds discipline cannot reach the experimental harness undetected.

3.3 Wire protocols and message formats

The wire protocol is the contract between server and client. Two realisations are defined: an optimized binary protocol carried over UDP datagrams, against which all positive claims are evaluated, and a baseline newline-delimited ASCII protocol carried over a reliable TCP byte stream. The two protocols carry the same five message types at the same cadences (§3.2.1) and differ only in their on-the-wire encoding and transport. Holding everything else equal is what permits chapter 4 to attribute observed differences to transport, serialisation, and the latency-hiding techniques of §3.6. The overview is in Table 3.1 as below.

Tag (hex)	Message	Direction	Size on the wire	Purpose
0x01	JOIN_REQUEST	client → server	1 B	request to join a session
0x02	JOIN_ACCEPT	server → client	2 B	reply with the assigned player id
0x03	INPUT_COMMAND	client → server	9 B	one input sample (seq, dx, dy, dt) at 60 Hz
0x04	WORLD_SNAPSHOT	server → client(s)	7 + 22 N B	broadcast authoritative state of N players at 20 Hz
0x05	DISCONNECT_NOTICE	client → server	2 B	best-effort notice of voluntary departure

Table 3.1: Optimized protocol message catalogue. Sizes include the one-byte type tag. N is the number of currently joined players. At the eight-player operating point a WORLD_SNAPSHOT is 183 B.

3.3.1 The optimized binary protocol

The optimized protocol defines five message types, each distinguished by a single-byte type tag carried as the first byte of every datagram. The tag has values one through five and permits the receiver to dispatch on type without committing to decode the rest of the payload, the property exploited by the receiver loops of §3.4 and §3.5 to discriminate between the input messages the server expects and the snapshot messages the client expects.

A *join request* is sent by a client immediately after binding its socket. It carries the protocol version the client was compiled against, on two bytes, so the server can refuse a session whose protocol it does not understand. Total: 3 bytes.

A *join accept* is the server's positive response. It carries the 16-bit player identifier the server has allocated and the two single-precision floats describing arena width and height. Total: 11 bytes. The arena dimensions are carried on the wire, rather than read by the client from the same shared

header as the server, in principle to permit arenas of arbitrary dimensions; here the value is fixed, and the field exists for symmetry with the baseline.

An *input command* is sent by a client at the input rate (60 Hz). It carries the player identifier, a per-player monotonically increasing 4-byte sequence number, and the two signed single-byte movement-intention coordinates of §3.2.1. Total: 9 bytes.

A *world snapshot* is sent by the server at the snapshot rate (20 Hz). It begins with the type of tag, followed by the 4-byte authoritative tick number and a 2-byte count of how many player records follow. Each player record is 22 bytes long: a 2-byte identifier, four single-precision floats for position and velocity, and a 4-byte field carrying the sequence number of the most recently integrated input for that player. The encoded snapshot size is therefore 7 + 22N bytes, ranging from 29 bytes for a one-player session to 183 bytes for the eight-player upper bound, well below any realistic Ethernet path's MTU, the property the protocol relies on to deliver each snapshot in a single datagram and avoid IP-level fragmentation and the cascading drops it would imply on a lossy path.

A *disconnect notice* is sent by the server to all remaining participants when a session ends. It carries the 16-bit player identifier of the departing participant. Total: 3 bytes.

The endianness discipline is uniform: all multi-byte integer fields are transmitted in network byte order, and all IEEE-754 single-precision floating-point fields are transmitted by re-interpreting their 32-bit representation as an unsigned integer and writing that integer in network byte order. The reinterpretation is byte-for-byte rather than arithmetic, so the protocol is independent of host endianness and identical bytes are emitted on hosts of either convention.

3.3.2 The baseline ASCII line protocol

The baseline implements the same five semantic message types as a newline-delimited ASCII protocol over a reliable byte stream. A join accept takes the form JA <player_id> <width> <height>\n; an input command, IN <seq> <mx> <my>\n; a snapshot, SS <tick> <count> followed by seven decimal fields per player (id, four position/velocity coordinates, last-processed-input sequence) and a terminating newline; a client disconnect, BYE\n.

It differs from the optimized in three small but jointly substantive respects. Records are variable-length and delimited in-band rather than by an out-of-band length, so the receiver must accumulate bytes into a holding buffer and scan for newlines before dispatching. There is no peek mechanism: type discrimination requires parsing the leading two-character token and is inseparable from at least a partial payload parse. Numerical fields are variable-length decimal strings, so per-message wire size depends on value magnitudes. The expected per-snapshot size is therefore an order of magnitude larger than the optimized upper bound, quantified empirically in §4.5.

The reliable byte-stream transport relieves the application of packet-boundary concerns at the cost of head-of-line blocking: a snapshot delayed by a single dropped TCP segment cannot be delivered until that segment is retransmitted, and any snapshot enqueued behind it is delayed by the same amount. This is the contrast the loss-sweep experiment of §4.4 is designed to expose.

3.3.3 The bandwidth-free acknowledgement

The 4-byte field at the end of every player-state record in every snapshot, on both protocols, carries the sequence number of the most recent input from that player which the server has already integrated. This single field, more than any other design decision in the wire protocol, is what permits the optimized client of §3.6 to perform reconciliation, and the project to perform measurement, both without any additional ACK traffic.

In its first role, it is the input from which the client's reconciliation procedure derives the boundary between inputs already accounted for by the authoritative state and inputs that must be replayed on top of it. Without it, the client would either need to assume a fixed processing latency and replay a fixed number of inputs (inaccurate under jitter) or the server would need to emit a separate per-input acknowledgement (doubling the upstream packet rate and introducing a new failure mode). The field eliminates both alternatives.

In its second role, it underpins the per-input acknowledgement-latency metric of §3.8. When the client receives a snapshot whose last-processed sequence is some value, it consults its own input history for the wall-clock time at which that input was emitted, and reports the difference, end-to-end, and free of clock skew because both timestamps are taken on the same machine.

In its third role, it underpins the perceived-delay metric of §3.8, the residual delay between the moment an input is emitted and the moment its visual effect resolves on the local screen. On the optimized path, prediction has already moved the local representation forward the moment the input was sampled, so perceived delay is substantially smaller than acknowledgement latency; on the baseline the two are essentially equal.

§3.8 elaborates the precise definition of both metrics; the present section's claim is only that the field on which both depend is supplied at no additional bandwidth cost.

3.3.4 Bounded encode and decode discipline

The encoders and decoders for every optimized message type are written against a small pair of bounded buffer abstractions. The writer refuses to write any field for which there is insufficient remaining capacity, and its refusal is sticky: subsequent writes also refuse, and the encoder returns length zero rather than a partially written buffer. The reader is symmetric. The world-snapshot decoder additionally rejects any player count above the eight-player upper bound regardless of buffer size, so that a malformed packet cannot drive an unbounded allocation. Together these guarantees discharge NFR-6 (robustness) and NFR-1 (determinism) and are exercised on every build by the self-test of §3.2.5.

3.4 The authoritative server

The server is the canonical owner of the simulation. Only it is entitled to declare what the position and velocity of every player actually are. Its responsibilities follow from this authority: advance the simulation at a fixed cadence regardless of how quickly inputs arrive, associate each input with a known participant and refuse out-of-order inputs or impostors, broadcast its authoritative view at a fixed cadence, and do all of this on a single host without admitting any race condition. The

components that discharge these responsibilities are described in the order in which they are exercised on a typical loop iteration.

3.4.1 The authoritative simulation

The simulation state is held in a single component which exposes a fixed-timestep tick procedure and a read-only collector that the snapshot path consumes. It holds an integer tick counter (incremented by exactly one on each tick) and a hash map from 16-bit player identifier to a per-player record. The per-player record holds the canonical player state of §3.2.1 together with two pieces of bookkeeping that exist only on the server: a pending movement intention (the two signed bytes most recently received from that player, with a flag indicating whether any input has yet arrived), and the highest sequence number ever seen from that player. New players are constructed with position fixed at the arena centre and both pending coordinates set to zero.

Inputs arrive through a single-entry point and are processed synchronously. The entry points first checks that the claimed player identifier corresponds to a known participant; if not, the input is silently dropped. It then compares the input's sequence number against the highest previously seen from the same player. A strictly older sequence is rejected, on the grounds that the player's intention has already been advanced past that point and re-applying it would manifest as the character oscillating between two positions on the path of a delayed duplicate. An equal sequence is accepted as idempotent (the case that arises when a duplicate datagram retries). A strictly greater sequence is accepted, the latest-seen counter is updated, and the per-player pending coordinates are replaced. The replacement values are clamped defensively to {-1, 0, +1}, so a peer sending an out-of-range byte cannot trick the simulation into super-speed motion. The accepted-sequence counter, the field carried back to the client in every snapshot, and the basis for the acknowledgement-free reconciliation of §3.3.3, is updated to the value of the accepted input.

A single tick iterates over all participants and, for each, normalises the pending intention into a unit vector (or the zero vector if no axis is requested), multiplies by the constant player speed of §3.2.1 to obtain the velocity, advances the position by velocity times the fixed time step, and clamps inside the arena rectangle. The kinematics are deterministic: two simulations stepped through the same sequence of accepted inputs produce identical positions and velocities, because

the only floating-point operations involved are a sign-or-zero per axis followed by a fixed multiplication and addition, neither of which introduces non-determinism beyond what IEEE-754 already prescribes for a single machine. This determinism is the property on which the client's prediction procedure of §3.6 ultimately depends: if the client's local kinematics agree byte-for-byte with the server's, then the only residual difference between the predicted and the authoritative state is the difference in their inputs at any given moment, which the reconciliation step is specifically designed to absorb.

A snapshot is constructed by an explicit collector that copies the canonical state of every participant into a vector and pairs it with the current tick number. The collector is a pure read of the simulation and is the only path by which simulation state crosses into the protocol layer.

3.4.2 Session lifecycle

The session manager turns the endpoint-keyed datagram traffic of the optimized path into the identifier-keyed simulation entities of §3.4.1. It holds two parallel maps: one indexed by normalised endpoint (§3.2.2) yielding the assigned player identifier and the timestamp of the most recent input from that endpoint, and one indexed by player identifier yielding the corresponding endpoint, so that the snapshot path can recover the recipient list without scanning.

A join request is processed idempotently: if the endpoint already has a session, the existing identifier is returned and the timestamp refreshed (the case when a client retries because the original was lost); otherwise the lowest unused identifier in {1..8} is allocated and both maps registered. If no identifier is available the caller signals session-full by simply not responding.

The session manager also authenticates inputs: every input carries the player identifier the client believes itself to have been allocated, and the verifier confirms it matches the identifier registered for the source endpoint. Failures are dropped, preventing one participant from supplying inputs that would be applied to another's character.

Liveness is enforced by a periodic reaping procedure called once per tick: it removes any session whose last-input timestamp is older than the configured 3 s timeout, together with the corresponding simulation entry. An explicit disconnect notice short-circuits the timeout via the same removal path.

3.4.3 Snapshot broadcaster

The snapshot broadcaster turns a collected snapshot into outgoing wire traffic. Its interface accepts a snapshot value and a list of recipient endpoints. Rather than encode once per recipient, it encodes the snapshot exactly once into a fixed 183-byte stack buffer (the upper bound of §3.3.1) and then iterates over the recipient list, transmitting the same bytes through the conditioner of §3.2.3 to each endpoint in turn. Per-snapshot encoding cost is therefore linear in N, not quadratic, the property that the bandwidth-scaling experiment of §4.6 relies on. Two cumulative counters are sampled by the periodic logger: total bytes pushed across all recipients (divided by recipient count to obtain a per-wire bytes-per-snapshot estimate), and the number of distinct broadcast events (used to verify the configured snapshot rate is honoured).

3.4.4 The optimized event loop

The components above are composed by a single-threaded event loop driven by a monotonic clock and a next-tick deadline. On each iteration it performs five things in sequence. It pumps the conditioner so any outbound packet whose scheduled delivery time has passed is in fact transmitted by the underlying socket. It drains the inbound socket of all matured packets, capped at 256 per iteration to prevent a flood from starving the tick. For each inbound packet it peeks the type tag and dispatches: register a session and reply with a join-accept, verify and apply an input, or process a disconnect; malformed or unknown packets are silently discarded. If the next-tick deadline has been reached it reaps timed-out sessions, advances the simulation by one tick, and (every third tick) broadcasts a snapshot. It then consults the periodic-log cadence and emits one row if a second has elapsed, and sleeps until the earlier of the next-tick deadline or 2 ms in the future.

The loop is robust to a tick spiral: if the next-tick deadline has fallen behind by more than one tick the deadline is snapped forward to the current time rather than allowed to accumulate as a burst. The duration governor that ends the process at a configured wall-clock elapsed time is checked once per iteration, so a scripted experiment exits within one iteration of its boundary.

3.4.5 The baseline server path

The baseline server is the deliberately-naive comparator. It honours the same simulation, time step, tick counter, and snapshot rate, and differs from the optimized path in three respects forced by the choice of a reliable byte-stream transport. It does not use the conditioner of §3.2.3: the conditioner operates on UDP datagrams, and the kernel will retransmit any TCP segment it attempts to drop. Conditioning is therefore applied at line granularity inside the baseline client (§3.5.4). It uses the ASCII encoder of §3.3.2 instead of the binary one. And it replaces the endpoint-keyed session manager with a hash map from connection file-descriptor to a per-connection record (player id, inbound holding buffer, last-input timestamp), accepting connections through a non-blocking accept loop and reaping closed ones on EOF.

Otherwise the baseline event loop is identical in shape to the optimized loop. Holding loop shape constant is what permits chapter 4 to attribute observed differences to transport, serialisation, and the latency-hiding techniques of §3.6.

3.4.6 Mode selection

The server accepts a small number of startup parameters that select between the two paths, configure the conditioner, and direct the periodic log to a labelled file under a chosen directory. Startup validates these, installs interrupt handlers so that an orchestration script can request an orderly shutdown, and dispatches to the optimized loop, the baseline loop, or both in sequence. In both-mode the optimized loop runs to completion before the baseline starts, avoiding port contention; this is the mode used by the main experiment matrix to gather an A/B pair from a single invocation. The shutdown signal is communicated through a single process-wide atomic flag that both loops poll once per iteration, sufficient on the server, which is single-threaded.

3.5 Client structure and rendering

The client is the participant-facing process. The same executable is launched once for each player; the difference between two clients in a session is reduced to the player identifier the server allocates at join time. The client must sample the participant's intention to move, transmit it at a fixed rate, receive the authoritative snapshots the server emits at its own fixed rate, and render the resulting state. The latency-hiding techniques that mediate between the slow snapshot cadence and the fast display rate, and that distinguish the optimized path from the baseline, are deferred to §3.6; the present section describes the scaffold inside which those techniques operate.

3.5.1 Process startup and configuration

The client accepts a small number of startup parameters: the server host, operating mode, run label for the local logs, optional duration, optional bot pattern, conditioner parameters (§3.2.3), and two ablation switches that suppress prediction or interpolation respectively. The ablation switches admit four distinct configurations of the optimized client and the chosen one is recorded as a mode label that flows through the log files into the analyzer. Startup validates the parameters, installs interrupt-signal handlers symmetric to the server's, and dispatches to the optimized or baseline client procedure.

3.5.2 Joining a session

The optimized client procedure binds a UDP socket to an ephemeral port, configures the conditioner, and emits a single join request. It then enters a blocking poll of at most two seconds, accepting only join-accept messages and ignoring everything else; on accept it records the assigned identifier and proceeds to the main loop, on timeout it exits with a non-zero status that the orchestrator reports as a failed harness setup.

Once joined, the client launches the network receiver thread (§3.6.1), opens its periodic and per-input log files (§3.8), constructs the predictor and interpolator, and creates the rendering window unless the run is headless. The window is suppressed automatically when a bot pattern is active.

3.5.3 The main loop

The main loop performs five activities per frame. It consumes accumulated window events. It drains all snapshots from the receiver-thread queue (§3.6.1), ingests their records into the authoritative state and (for remote players) the interpolation buffer, performs reconciliation against the local record (§3.6.3), and uses the acknowledged sequence number to retire pending input samples and emit a per-input log row. It samples the input source at 60 Hz, advances the local prediction by one step, transmits the input to the server, and registers a pending sample so the next snapshot can compute its acknowledgement latency. It composes a render list from the local predicted position and the interpolated remote positions, hands it to the renderer, and overlays the head-up display. It samples the periodic log once per second.

The loop is bounded by a configured wall-clock duration or a window-close event. On exit it transmits a best-effort disconnect notice, stops the receiver thread, flushes both logs, and returns.

3.5.4 Input: keyboard, focus gating, and the bot harness

The input source is parameterised on a small abstract interface that produces a movement-intention sample per call. Two implementations are defined: a keyboard reader, mapping the cursor and WASD clusters onto the two signed-byte axes of §3.2.1, and the bot harness.

The keyboard implementation includes a non-obvious correctness guard. The graphics library's keyboard query is a global OS poll: it reports the host keyboard state as a whole, not as filtered by which window has focus. Naive use, in a process running alongside sibling clients on the same host (the configuration of the manual two-player demo and several experiments), produces the symptom that every keystroke is received by every client and all participants move in lockstep. The guard consults the window's focus status before any polling and returns an empty intention if the window is unfocused; a short head-up-display message informs the participant when input is being suppressed for this reason. The same guard applies on the baseline. This is one of the deviations recorded in the project's deviations log.

The bot harness produces intentions algorithmically, so experiments can be driven without human attention and replications with the same seed produce identical input traces. Four patterns are

defined: a circular loop around the arena centre, a sinusoidal oscillation on one axis, a seeded random pattern picking a new direction at fixed intervals, and a zigzag rotating through the cardinal directions on one-second boundaries. Each bot is seeded from a base seed combined with the local player identifier, so distinct participants in a session do not synchronise their motion while two replications of the same session do.

3.5.5 Rendering and head-up display

The renderer is a thin layer over the graphics library. It owns a 1280 × 720 window capped at 120 fps and a single cached font (the HUD degrades cleanly if no font can be loaded). The render pass clears, draws the arena rectangle, draws each player from a render list as a coloured circle of the §3.2.1 radius (colour deterministically selected from a fixed eight-entry palette indexed by player id), and overlays the HUD. The local participant carries a white outline; a runtime-toggleable ghost overlay draws its authoritative position translucently, separating from the predicted position under sustained reconciliation. The HUD reports mode, identifier, snapshot/input totals, latest acknowledgement-latency and reconciliation samples, interpolation buffer depth, runtime-toggle state, and the focus flag. F1-F4 bind runtime toggles for HUD visibility, the ghost overlay, prediction, and interpolation; the experimental harness instead uses the startup ablation switches of §3.5.1, so the mode flowing into the logs is fixed for a run. The following figure is a screenshot of the render and the HUD.

Figure 3.2: The optimized client at runtime in a two-player session under the B_lat100 condition. The local participant is the white-outlined circle (lower right); a remote participant is rendered at its 100-ms-delayed interpolated position (upper

left). The translucent overlay attached to the local player is the authoritative "ghost" enabled with F2; in this frame the predictor and the most recently received authoritative snapshot agree to within a pixel. The head-up display (top-left) reports mode, player id, snapshot/input totals, latest acknowledgement-latency and reconciliation samples, interpolation buffer depth, and the runtime toggle states described in §3.5.5.

3.5.6 The baseline client path

The baseline client mirrors the optimized client but omits everything the baseline transport disallows or the absence of the §3.6 optimizations makes redundant. It opens a TCP connection and reads the join-accept line emitted as the first line of the stream. Its main loop has the same shape as the optimized loop but with three structural differences.

It applies its own line-granularity conditioning inside the client itself (two priority queues, outbound and inbound), since the §3.2.3 conditioner is defined on UDP datagrams and would be defeated by TCP retransmission. A line on which the loss roll succeeds is discarded. The two conditioners share their seed with the optimized conditioner of the same experiment, so A/B-paired runs are driven by identical pseudo-random sequences.

It uses neither predictor nor interpolator: the baseline's local view of any participant, including the local one, is exactly the most recent authoritative state received. The local participant therefore lags its own input by the network round-trip plus one snapshot period, the cost the headline figure of §4.2 summarises.

It uses no receiver thread, performing socket reads on the rendering thread and parsing newline-delimited lines on each iteration. The renderer, HUD, input subsystem (with the same focus gate), and bot harness are shared between the two paths. This structural sharing is what makes the chapter-4 comparison one of network and latency-hiding strategies, rather than of two unrelated codebases.

3.6 Client-side latency hiding

The two preceding sections established the structural plumbing of the optimized client. The present section describes what the predictor and the interpolator actually do, the techniques that distinguish the optimized path from the baseline and that the project's positive claims of chapter 4 are claims

about. The section is organised in the order data flows through the client: receiver thread (§3.6.1), then predictor and reconciler for the local participant (§3.6.2-3), then interpolator for remote participants (§3.6.4), and the ablation surface (§3.6.5).

3.6.1 The network receiver thread

The optimized client runs a dedicated background thread that takes inbound snapshot datagrams off the network and hands them to the main loop through a thread-safe queue. It is launched after the join procedure of §3.5.2 succeeds and runs until orderly shutdown.

Its body is a tight loop. It pumps the conditioner of §3.2.3 so that any datagram whose scheduled delivery time has passed is in fact released, attempts a single non-blocking receive, and sleeps for one millisecond before retrying if no data is available. When data is available it peeks the type tag and decodes only world snapshots; every other type is ignored on this thread, since handling it would mutate non-thread-safe main-thread state. The decoded snapshot is stamped with the current wall-clock time and pushed onto the queue.

Taking the timestamp on the receiver thread, not the main thread, is a deliberate decision. The interpolator (§3.6.4) drives its render-time offset against this timestamp, and the per-input log derives acknowledgement latency from it; both metrics measure true wire time only if the timestamp is taken at the moment the datagram crosses out of the conditioner, before any main-thread queueing latency has been added. A main-thread timestamp would conflate network latency with the wait for the next rendering frame.

The queue itself is a mutex, a deque of stamped snapshots, and a condition variable; the receiver appends and signals, the main thread polls non-blockingly each frame. The condition variable is presently unused but preserves the option to move to a blocking pop later. A separate atomic counter publishes the running byte total for the HUD.

3.6.2 Client-side prediction

The predictor allows the local participant's character to respond to input with no perceptible delay. Without it, the participant's input would be transmitted to the server, integrated on the next tick, broadcast in the next snapshot, and only then visualised; the resulting input-to-visible delay would be the sum of the network round-trip and one snapshot period, roughly 150 ms on the 100 ms round-trip operating point of chapter 4. With the predictor enabled, the local character begins moving in the same render frame as the input is sampled, and the delay is bounded below by a single rendering frame.

The predictor maintains a fixed-size ring buffer of the most recent 120 inputs sampled by the participant, at the input rate of 60 Hz, two seconds of history. Each entry holds the input's monotonically increasing sequence number, its two signed-byte movement intentions, and the per-frame time-step at which it was sampled. A ring buffer rather than an unbounded queue bounds memory; entries older than two seconds will in any realistic operating point already have been acknowledged.

The predictor's central operation records an input and advances the local position in a single atomic step. When the main loop samples a new input, it calls into this operation with the two intentions, the time-step, and a reference to the local position. The operation assigns the next sequence number, appends the input to the ring, advances the local position by the same kinematic step the server's authoritative tick (§3.4.1) would apply, and returns the assigned number. The caller uses that number both as the wire-protocol field on the outgoing input message and as the key under which it registers the input in its pending-input table for the acknowledgement-latency measurement.

Crucially, the kinematic step inside the predictor and the kinematic step inside the server's authoritative tick are byte-for-byte identical: they call the same shared helpers on the same input values with the same floating-point arithmetic. This is the property that makes prediction *predictive* in the strict sense that, in the absence of new authoritative information, the predicted position would converge with the eventual server-broadcast position to within a single floating-point ULP. Any divergence observable at reconciliation time is therefore attributable to a real difference of state, most commonly an input the server has not yet processed, or a snapshot the client has not yet received, rather than to subtle arithmetic disagreement between two parallel implementations of what is supposed to be the same kinematic law.

The predictor exposes one further accessor that the per-input logging path (§3.8) uses: a lookup by sequence number that returns the recorded input if it is still in the ring. This is the mechanism by which an arriving acknowledgement is matched against the send-time of the input it acknowledges; the predictor itself maintains no timestamps.

3.6.3 Server reconciliation

The reconciliation procedure absorbs an arriving authoritative snapshot without discarding the predicted position the participant is currently looking at. It runs on the main thread once per snapshot, immediately after the snapshot has been ingested into the local view of authoritative state, and only on the local participant's record; remote participants are handled by the interpolator.

The procedure has two phases. Snap-to-authoritative overwrites the local position with the position carried in the snapshot. Replay then iterates the predictor's input ring chronologically, and for every entry whose sequence number is strictly greater than the acknowledgement field on the snapshot, applies the same kinematic step the server would apply. After both phases the local position is, by construction, what it would be if the server were to receive every input the client has so far sent and integrate them all in order, including those it has not yet received.

Chronological order of replay matters: the kinematic step is not commutative under the arena clamp, since the clamp truncates the trailing component of a velocity that would otherwise carry the position outside the arena. Replaying in chronological order is the only order that reproduces the server's eventual position; any other order would introduce a systematic bias in exactly the cases that benefit most from prediction, those in which the participant is moving along a wall.

The procedure also computes a runtime self-check. The local position is captured into a temporary before snap-to-authoritative; after replay, the Euclidean distance between the temporary and the new local position is the reconciliation error in pixels, the magnitude of the visual discontinuity reconciliation has just imposed on the local character. In a noiseless operating point it is zero to within floating-point rounding, because the server has correctly processed every input sent and the predicted position is exactly the position the server has just confirmed. In a noisy operating point it is positive and bounds from above the visual disturbance the participant could perceive. The per-snapshot reconciliation error is recorded in the per-input log; the analysis pipeline uses it as a sanity

check on the predictor, were it ever to grow without bound, predictor and server tick would have disagreed on the kinematic step and the bundled comparison would be uninterpretable.

When the participant has elected at startup to disable prediction, neither phase is applied: the local position is replaced wholesale by the authoritative position. The predictor's ring is still maintained, but only so that outgoing input sequence numbers remain monotonic and the per-input log can still match acknowledgements against send-times. The local character then lags its own input by the network round-trip plus one snapshot period, identical to the baseline.

3.6.4 Entity interpolation

The interpolator is the dual of the predictor. Where the predictor hides latency on the local character at the cost of a small reconciliation correction on every snapshot, the interpolator hides the discreteness of the snapshot stream on the remote characters at the cost of rendering them slightly in the past.

For every remote participant whose records have been ingested it maintains a bounded history of the most recently received snapshot samples. Each entry holds a position, a velocity, and the wall-clock time at which the snapshot was received by the receiver thread. The history is capped at 32 entries, at 20 snapshots per second, slightly more than 1.5 seconds, sufficient to span any short snapshot drought introduced by jitter or loss, and small enough that the linear search on it is negligible in the per-frame budget.

On every rendering frame, the renderer asks the interpolator for the position at which to draw each remote participant. The interpolator computes a target time of now minus 100 ms, searches the participant's history for the pair of consecutive entries whose receive-timestamps bracket the target, and returns the linear interpolation of their two positions. If the target falls before the earliest entry the interpolator returns the earliest known position; if it falls after the most recent entry (during a snapshot drought) it returns the most recent known position rather than extrapolating beyond it.

The 100 ms render-time offset is the project's most consequential tuning parameter and is fixed by two competing pressures. It must be strictly larger than the snapshot inter-arrival interval (50 ms at 20 Hz) so the target time falls between two real entries rather than past the most recent one, with

margin for jitter. It is also the additional delay with which any remote participant's motion is rendered relative to its authoritative time; a participant delayed by half a second is no longer playing the same game. The chosen 100 ms is large enough to bridge two missing snapshots in succession (a sustained 10 % loss rate), small enough that the additional delay is imperceptible.

The decision not to extrapolate during a drought is deliberate and is the source of one of the most visible behavioural differences between the two paths in chapter 4. A client that extrapolated by carrying the most recently known velocity forward would produce smoother short-term motion but would have to retract it when the next snapshot revealed that the remote participant had turned or stopped, the rubber-band artefact. The interpolator instead holds the last known position, producing a brief stutter rather than a snap-back; the stutter is what the snapshot-arrival-gap percentile metric of §4.3 quantifies.

A staleness bound completes the picture: if a participant's most recent entry is older than two seconds, the participant is judged to have disappeared and a periodic prune reclaims the storage. The bound matches the server's session-manager bound (§3.4.2), so the remote participant disappears from the local view at very nearly the same instant the server reaps them.

When the participant has elected at startup to disable interpolation, the remote participant is drawn at the position carried in the most recently received snapshot. The visual signature of this configuration is a 50 ms stairstep on every new snapshot, which the optimized-without-interpolation condition of §4.3 demonstrates.

3.6.5 The ablation surface

Two boolean startup switches on the optimized client suppress prediction and interpolation respectively; together they admit the four configurations introduced in §3.5.1. Each is recorded as a mode label flowing through every row of the per-input log: *optimized*, *optimized_noPred*, *optimized_noInterp*, and *optimized_raw*. The same client executable, renderer, conditioner, and baseline are exercised in every configuration; only the two switches differ.

The mode label is the key on which the analyzer (§3.10) partitions the per-input data when it generates the isolation-matrix figure of §4.3, the figure that decomposes the cumulative perceived-delay improvement of the bundled optimized path into the contributions of its constituent techniques. Without an ablation surface, claims of the form "of the n-millisecond improvement, m milliseconds are attributable to prediction" are conjecture.

Runtime toggles bound to function keys (§3.5.5) allow the same techniques to be switched on and off interactively, but they do not update the mode label; the experimental harness uses the startup switches exclusively.

3.7 Concurrency model

The two preceding pairs of sections described the server and the client as collections of components organised around the data flow of a single tick or a single frame. The present section re-presents the same material from a different angle: which components run on which thread, what synchronisation primitives mediate the interactions, and what guarantees the resulting structure preserves.

3.7.1 The server is single-threaded

The server process, in both modes, runs the entire event loop of §3.4 on a single thread. Pumping the conditioner, draining the inbound socket, advancing the simulation, broadcasting snapshots, and emitting periodic log rows are interleaved sequentially within one thread, with no second thread the loop's state has to be defended against. The baseline server has the same property.

The decision is a deliberate simplification. At the operating points exercised in chapter 4, up to eight participants, 60 ticks per second, 20 snapshots per second, at most 183 bytes per snapshot, the per-iteration work is comfortably bounded below the 16.67 ms tick budget, and adding a thread would have introduced a synchronisation surface around the simulation state in exchange for no measurable improvement. The bottleneck lies in per-snapshot bandwidth and simulated network

latency, neither helped by additional server-side parallelism. The only shared piece of state the loop reads without holding a mutex is the atomic shutdown flag of §3.7.5.

3.7.2 The client uses two threads

The client process runs on two threads. The main thread runs everything described in §§3.5 and 3.6, event poll, snapshot drain, reconciliation, input sample, log emission, interpolated rendering. The receiver thread of §3.6.1 receives, decodes, and timestamps inbound snapshot datagrams and pushes them onto the snapshot queue.

The bot harness, when active, runs synchronously inside the input-sample branch of the main loop, by deliberate choice: a separate thread for the bot would either need to be polled by the main loop or push inputs onto a cross-thread queue, introducing a synchronisation surface the project does not need.

The baseline client of §3.5.6 starts no receiver thread. Its inbound and outbound socket activity and its rendering all happen on the same thread; this is sufficient because its per-frame work is dominated by rendering rather than snapshot ingestion. The optimized client's two-thread structure exists specifically to remove the rendering frame's contribution from the inbound snapshot's perceived latency, the property on which the receive-side wall-clock stamp of §3.6.1 depends.

3.7.3 Synchronisation primitives in use

Three standard-library primitives appear in the codebase, each with a narrow and consistent use.

A *mutex* appears in three places: guarding the conditioner's pending queue and PRNG (§3.2.3); guarding the snapshot queue's deque on the optimized client; and guarding the file pointer inside the thread-safe CSV sink of §3.2.4. In every case the mutex protects exactly one short critical section that mutates exactly one container.

A *condition variable* appears once, inside the snapshot queue. It is signalled by the receiver thread on every push, but the present main loop polls the queue once per frame rather than block on it (since the main thread has rendering work to do regardless). It is retained because retaining it costs

nothing and a future revision could move to a blocking-wait main loop without changing the queue's interface.

Atomic flags (boolean shutdown flags of §3.7.5, the receiver thread's run flag) and an *atomic 64-bit counter* (the running byte total) complete the set. Both are used with their default sequentially-consistent ordering; the simpler reasoning that sequential consistency permits is worth its negligible cost at these access rates.

3.7.4 The snapshot queue is the only cross-thread data path

On the client, the snapshot queue and the receiver thread's two atomics together constitute the entire cross-thread data surface. Every other piece of state, predictor's input ring, interpolator's history, local position, pending-input table, HUD record, rendering window, log file pointers, keyboard sampler, bot, lives entirely on the main thread.

The snapshot queue is a strict single-producer single-consumer channel: exactly one writer (the receiver, which only pushes) and exactly one reader (the main thread, which only pops). The producer and consumer cannot mutate the same deque entry simultaneously, because the two ends of a non-empty deque are distinct entries and an empty deque rejects the consumer's pop before the lock is even released.

This narrow surface is what makes the project's multi-threaded code reasoning-tractable: no thread holds more than one mutex at a time (no deadlock cycle), the consumer does not wait (no lost wake-up), the byte counter is only ever loaded by the consumer (no stale-read concern). The set of concurrency bugs the model could in principle exhibit is correspondingly small.

3.7.5 Shutdown coordination

Both processes install a handler for the standard interruption signal (and on the server for the standard termination signal as well). The handler does only one thing: it stores *true* into a process-wide atomic flag. The main loop polls the flag once per iteration and exits the first time it observes it true.

On the server, orderly shutdown flushes and closes the periodic log and prints a single status line; there is no other thread to coordinate with.

On the client, orderly shutdown transmits a single best-effort disconnect notice, signals the receiver thread to stop by clearing its run flag, joins the receiver (which observes the flag on its next iteration after waking from the 1 ms backoff), flushes both log files, and closes them. If the disconnect notice is dropped or delayed by the conditioner, the server's liveness reaper of §3.4.2 will remove the participant within three seconds.

The atomic-flag mechanism is the only point at which the operating system's asynchronous event delivery interacts with the synchronous event loop of either process. Its narrowness lets the project support both externally orchestrated shutdown (every chapter-4 experiment) and manually triggered shutdown (interactive demonstrations) through a single mechanism.

3.8 Instrumentation and observability

The system has been described to this point in terms of what it does. The present section describes what it measures about itself. The instrumentation is the bridge between the methodology of this chapter and the experimental results of the next: every figure in chapter 4 is generated from a comma-separated-value file emitted by a running process, and every claim in chapter 4 is a claim about a column of one of those files.

There are three log files per run. The server emits one time-series file, sampled once per second. Each client emits two: a similar time-series file sampled once per second, and an event-driven file with one row per acknowledged input. The two cadences are complementary: the time-series cadence is sufficient for any quantity that varies on the timescale of seconds (cumulative bytes, participant count, current interpolation buffer depth), while the event cadence is necessary for any quantity defined per input (acknowledgement latency, perceived delay, reconciliation error). Both

cadences are flushed to disk every second, so a process killed mid-experiment loses at most the last second of data.

3.8.1 Acknowledgement latency versus perceived delay

The most consequential decision in the project's instrumentation is to record two distinct latency metrics rather than one. Conflating them, a common error in informal write-ups of client-side prediction, would obscure the very effect that the project is designed to measure.

The **acknowledgement latency** is the wall-clock interval between the moment an input is transmitted by the client and the moment the server's acknowledgement of that input first appears on the client. The client records the wall-clock send time of every input it transmits, and on every arriving snapshot it consults the acknowledgement field of section 3.3.3 to identify the most recent input the server has processed; the difference between the current wall-clock time and the recorded send time is the acknowledgement latency for that input. Because both timestamps are taken on the same machine, the metric is end-to-end and free of any clock skew between the two processes. It is logged under the column name **rtt_ms**.

The **perceived delay** is the wall-clock interval between the moment the participant produces an input and the moment the visual effect of that input appears on the participant's screen. On the optimized path with prediction enabled, the predictor advances the local position before the input is even transmitted, so the perceived delay is bounded above by a single rendering frame, which the project records as a lower-bound sample of zero. On the baseline, and on the optimized path with prediction disabled, the visual effect is not produced until the snapshot carrying the acknowledgement arrives, and the perceived delay equals the acknowledgement latency. The metric is logged under the column name **perceived_delay_ms**, and it is this column, not the acknowledgement latency, that the headline figure of section 4.2 plots. The two metrics are equal for the baseline and substantially divergent for the optimized path; recording both is what makes the difference visible.

3.8.2 The three logs

The **server time-series log** carries six columns: a wall-clock sample time, the current authoritative tick, the connected participant count, the cumulative bytes sent, the cumulative snapshots emitted, and the cumulative inputs received. The byte and snapshot totals are the only direct measurements of network traffic taken anywhere in the project; every bandwidth-related figure in chapter 4 is derived from them.

The **client time-series log** carries seven columns: a sample time, the local participant identifier, the most recent authoritative tick observed, the cumulative bytes received from the conditioner, the cumulative inputs sent, the most recent acknowledgement-latency sample, and the current interpolation buffer depth. The cumulative byte total is the figure from which per-client bandwidth in kilobits per second is derived (by differencing successive samples); the interpolation depth column confirms the interpolator is populated; the acknowledgement sample is a low-rate mirror of the per-input file's **rtt_ms** and serves only as a sanity check.

The **per-input event log** is the densest and most consequential of the three. The client emits one row per pending input that an arriving snapshot's acknowledgement field has just retired. Each row carries eight columns: the wall-clock emission time, the participant identifier, the input's sequence number, the snapshot's acknowledgement field, **rtt_ms** and **perceived_delay_ms** as defined in section 3.8.1, the reconciliation error in pixels defined in section 3.6.3, and the mode label of section 3.6.5. The mode label is the partition key on which the analyser separates the four optimized configurations in the isolation experiment of section 4.3; every other column is a metric. At sixty inputs per second, a one-minute run produces roughly three thousand six hundred rows per participant, which is enough to estimate any percentile of any metric usefully.

A further metric, the per-snapshot **arrival gap**, is computed not from the metric columns of this file but from differences between successive wall-clock timestamps within a single mode-tagged

participant series. It is the project's quantitative measure of remote-motion smoothness and is the column on which figure 3b of section 4.3 is built.

3.9 Build system and reproducibility

The project is reproducible from a clean checkout in three commands: configure, build, test. The present section describes the small amount of build-system machinery that makes that claim true, and the deviations log that records every place where the implementation has departed from the original specification.

3.9.1 Build configuration

The build is driven by CMake against a C++20 toolchain. The project requires a single external dependency, the SFML 3 graphics, window, and system libraries, and depends on the standard threading library through the build system's portable threads target. On Apple platforms the build adds a Homebrew-default search path so that the SFML installation is found without per-developer configuration; on Linux the distribution's package-manager installation is found through the build system's default search.

The build produces three executables. The first is the server, which links the small shared static library that contains the serialisation, conditioner, logging, and socket-compatibility code of section 3.2. The second is the client, which links the same shared library together with the SFML libraries. The third is the self-test target of section 3.2.5, which links only the shared library and is registered with the build system's test runner so that the project's correctness contract on the wire format is exercised by a single command.

The build defaults to release configuration with full optimisations and warnings-as-extras enabled. A debug configuration is supported but is not used for any experimental data reported in chapter 4.

3.9.2 Repository layout

The repository is organised by responsibility, not by class. The `common/` directory contains the shared substrate of section 3.2 and the wire format of section 3.3. The `server/` and `client/` directories each contain the entry point, the event loop, and the components specific to that process. The `tools/` directory contains the orchestration scripts and the analysis pipeline of section 3.10. Experiment outputs are written to dedicated top-level directories that are excluded from version control. Every artefact in the repository is either source, configuration, or documentation; nothing is generated and committed.

3.10 Evaluation pipeline

The evaluation pipeline is the layer that drives the system through the operating points reported in chapter 4 and turns the resulting log files into figures. It is the project's discharge of the reproducibility requirement NFR-2: every figure in chapter 4 is produced by the pipeline from raw inputs that the pipeline itself emits, with no manual intervention at any step. The present section describes the four experiment matrices the pipeline runs, the analysis program that consumes their output, and the across-replication aggregation that turns per-run measurements into the confidence-interval-adorned points that chapter 4 plots.

3.10.1 The four experiment matrices

The pipeline drives the system through four orchestrated experiment matrices. The **main** matrix is the original five-condition-by-two-mode comparison and is the matrix from which the headline figures of section 4.2 are derived. Its five conditions span the operating points the project most cares about: a clean condition with no added latency or loss, two latency conditions at one hundred and two hundred milliseconds round trip, a combined latency-and-loss condition, and a stress condition with eight participants. The two modes are the optimized and baseline paths of section 3.4.

The **isolation** matrix exercises the four optimized configurations of section 3.6.5, plus the baseline, at a single operating point. It is the matrix from which the isolation-decomposition figure of section 4.3 is derived; its purpose is to attribute the bundled improvement to its individual constituent techniques.

The **loss sweep** matrix runs both modes at five loss percentages between zero and ten, with all other parameters held at the same operating point as the latency-only condition of the main matrix. It is the matrix from which the loss-tail and snapshot-jitter figures of section 4.4 are derived.

The **scale sweep** matrix runs both modes at five participant counts between one and eight, with no added latency or loss. It is the matrix from which the bytes-per-snapshot and bandwidth-scaling figures of section 4.6 are derived.

Each matrix is implemented as a small shell orchestrator that launches the server in the background, launches the configured number of clients in the foreground (each driven by one of the four bot patterns of section 3.5.4), waits for the configured duration to elapse, and reaps the participating processes. A dedicated subdirectory is created per run, named for the condition, the mode, and the replication index, and the run's log files are written into it. A single one-shot script runs all four matrices in sequence and then invokes the analyser on each, producing the complete report figure pack from a single command.

3.10.2 The analysis program

The analysis program is a single Python script that operates in one of four modes corresponding to the four matrices. Each mode reads the raw log files from the matrix's output directory, partitions the runs by the matrix's natural keys (condition and mode for the main matrix, mode label for the isolation matrix, loss percentage for the loss sweep, participant count for the scale sweep), and emits two artefacts per matrix: a comma-separated-value summary table with one row per partition cell, and a fixed set of figures in PNG form.

The summary tables are the canonical output for any reader who wants the underlying numbers without re-deriving them. The figures are the artefacts that chapter 4 reproduces.

The analysis program reads only the columns introduced in section 3.8. Per-input metrics are aggregated across the rows of the per-input log; per-second metrics are aggregated across the rows of the time-series log; cumulative metrics are differenced across the time-series log to recover the underlying rate. The arithmetic involved in each derivation is deliberately elementary, so that any calculation in chapter 4 can be reproduced with a spreadsheet against the same raw log files.

3.10.3 Replications and confidence intervals

Each cell of each matrix is run three times by default, with a deterministic per-replication seed. The seed is propagated all the way through to the in-application network conditioner of section 3.2.3, so that two replications of the same cell are driven by the same pseudo-random latency and loss sequences as each other only at the level of the conditioner; the small remaining sources of run-to-run variation come from operating-system scheduling, from the wall-clock time at which each replication happens to start, and from the resulting subtle differences in how the periodic and per-input cadences interleave with the conditioner's pumps.

For each cell, the analyser computes a per-replication summary metric (the mean of the per-input perceived delay, for example), then aggregates across the replications by computing the across-replication mean and the standard error of the mean. A ninety-five-percent confidence interval is derived from the standard error using the standard-normal multiplier of one-and-ninety-six-hundredths, which is the appropriate approximation given the scale of each per-replication mean (the mean of several thousand per-input rows) and the small-sample replication count. Cells with only one replication are reported with a zero-width interval and a flag that the analyser propagates into the figure caption, so that no such cell is silently treated as if it had been adequately sampled.

Every chapter-4 figure that compares two or more cells is plotted with these confidence intervals visible: as error bars on bar charts, as shaded bands around line plots, and as explicit columns in any tabular comparison. The intervals are the project's quantitative statement of “do not over-interpret small differences”; if two cells overlap on their intervals, the report does not claim a difference between them. Where the intervals do not overlap, the difference is reported as significant at the conventional five-percent level. The discipline is uniform across all four matrices.

The same aggregation procedure is applied to every metric without exception: the perceived delay, the acknowledgement latency, the per-client bandwidth, the snapshot-arrival coefficient of variation, the per-snapshot byte count, and the arrival-gap percentiles of section 3.8.4 are all computed, across-replication averaged, and confidence-bounded by the same code path.

Chapter 4. Experiments and Results

4.1. Experimental setup

4.1.1 Platform

All measurements were taken on a single workstation running macOS on Apple Silicon. The server, every client, and the network conditioner ran as separate processes on the same host, communicating through the loopback interface. No measurement in this chapter exercises a real link; every reported latency, loss event, or bandwidth figure is the result of the in-application conditioner described in chapter 3, applied on top of a transport whose own contribution to latency and loss is, on loopback, well below the resolution of the metric. The implications of this choice, most importantly the fact that the wire protocols are not being asked to defend against the kind of impairments a wide-area path would impose, are discussed in section 4.8.

4.1.2 Build configuration

The C++ codebase was compiled in its release configuration with optimisations enabled, against the C++20 language standard, using the system toolchain on the test platform. The graphical client was linked against version 3.0.0 of the SFML library; this is the version against which the migration described in chapter 3 was completed. Bot-driven clients were built from the same sources as interactive clients and differed from them only in the input source they instantiated. The server binary used in the optimised configuration was the same binary used in the baseline configuration, with the wire protocol selected at start time by a single command-line switch. This guarantees that no measured difference between the two modes can be attributed to a difference in compiler flags or linker layout.

4.1.3 Workloads

Every measurement in this chapter used the same four deterministic bot workloads, circular motion, sinusoidal motion, pseudo-random motion, and a zig-zag pattern, described in chapter 3. The bots emit input commands at the same rate as a hypothetical human player would (sixty samples per

second) but along trajectories that are reproducible from a seed and that do not depend on operator skill, fatigue, or attention. This makes the measured outputs a property of the system rather than of the operator, and it is the reason every condition can be replicated three times and averaged with confidence intervals that are meaningful. For experiments that involve more than one client at a time, the bots used were chosen from this set in round-robin order so that no two adjacent clients were following identical trajectories.

4.1.4 Conditioner parameters

The added round-trip time, jitter, and packet-loss probability that appear in the names of the experimental conditions refer to parameters supplied to the conditioner described in chapter 3, not to properties of the underlying loopback transport. Round-trip time is reported as the additional latency the conditioner imposes on top of the loopback baseline, applied symmetrically in both directions. Jitter is a uniform perturbation on top of that base latency. Loss is the per-packet drop probability, sampled independently for each packet from a seeded pseudo-random generator, and applied symmetrically on the optimised path and asymmetrically on the baseline path; the consequences of this asymmetry are addressed honestly in section 4.8. The seed used by the conditioner was the same across the optimised and baseline runs of a given replication, so that within a replication the two modes faced the same sequence of jitter offsets and loss draws.

4.1.5 Run length, replication, and aggregation

Unless stated otherwise, every cell in every experimental matrix was executed for thirty seconds of wall time and replicated three times. Thirty seconds was chosen as a balance between two competing pressures: shorter runs do not accumulate enough samples for the tail percentiles to be stable, while longer runs lengthen the overall evaluation cycle to a point that discourages re-running it after every change. Three replications per cell is the smallest replication count for which a sample standard deviation is meaningful; it is not enough to support strong claims about the *shape* of the per-replication distribution, but it is enough to attach a 95 % confidence interval to a mean using the normal approximation described in chapter 3. Larger run lengths and replication counts are supported by the same harness, the relevant knobs are exposed as

environment variables, and were used during development to confirm that the headline conclusions are not artefacts of the default configuration.

Across all four matrices reported in this chapter, the main matrix, the isolation matrix, the loss sweep, and the scale sweep, every figure is a plot of the across-replication mean for the metric in question, with error bars representing the 95 % confidence interval where the figure type permits. Where a figure presents a percentile rather than a mean, the percentile is computed over the pooled sample from all replications of the cell rather than the mean of per-replication percentiles; this is a conservative choice that gives the tail more samples to be estimated from, at the cost of slightly weakening the independence assumption that the confidence intervals would otherwise rest on. The decision is recorded here so that any reader who wishes to recompute the percentiles in a different way can do so from the raw per-run files, which are preserved alongside the summary tables.

4.2. Headline result: wire delay versus perceived delay

4.2.1 The wire view and the perceived view

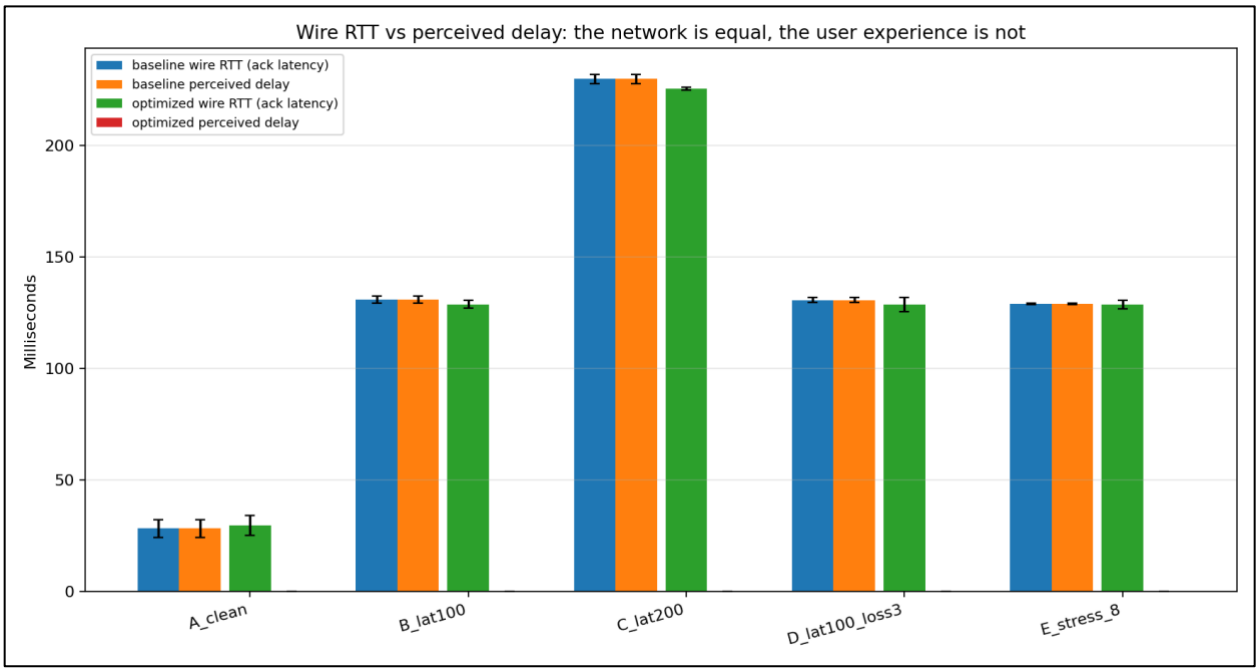

Figure 4.1 plots, for each of the five conditions of the main matrix, the across-replication mean wire round-trip time and the across-replication mean perceived delay, for both the optimised and the baseline mode. Error bars are the 95 % confidence intervals of section 3.10.3.

In Figure 4.1, two facts are visible immediately. *First*, within each condition the wire round-trip time is statistically the same for both modes: 29 ms vs 28 ms in the clean condition, 129 ms vs 131 ms at 100 ms injected RTT, 226 ms vs 230 ms at 200 ms, and similar agreement in the loss and stress conditions. The conditioner of section 3.2.3 imposes the latency on both modes from the same seed, so this equivalence is by construction; its presence in the data is the precondition that gives the rest of the chapter its credibility. *Second*, the perceived bars diverge sharply. On the baseline they equal the wire bar in every condition, because the baseline client renders only authoritative state. On the optimised path they collapse to zero, because the predictor of section 3.6.2 advances the local representation in the same render frame as the input is sampled. At 200 ms RTT the gap between the two perceived bars is therefore the entire 230 ms of measured wire latency.

4.2.2 Perceived responsiveness across conditions

Figure 4.2 redraws the same data under a perceived-only lens, omitting the wire bar for clarity.

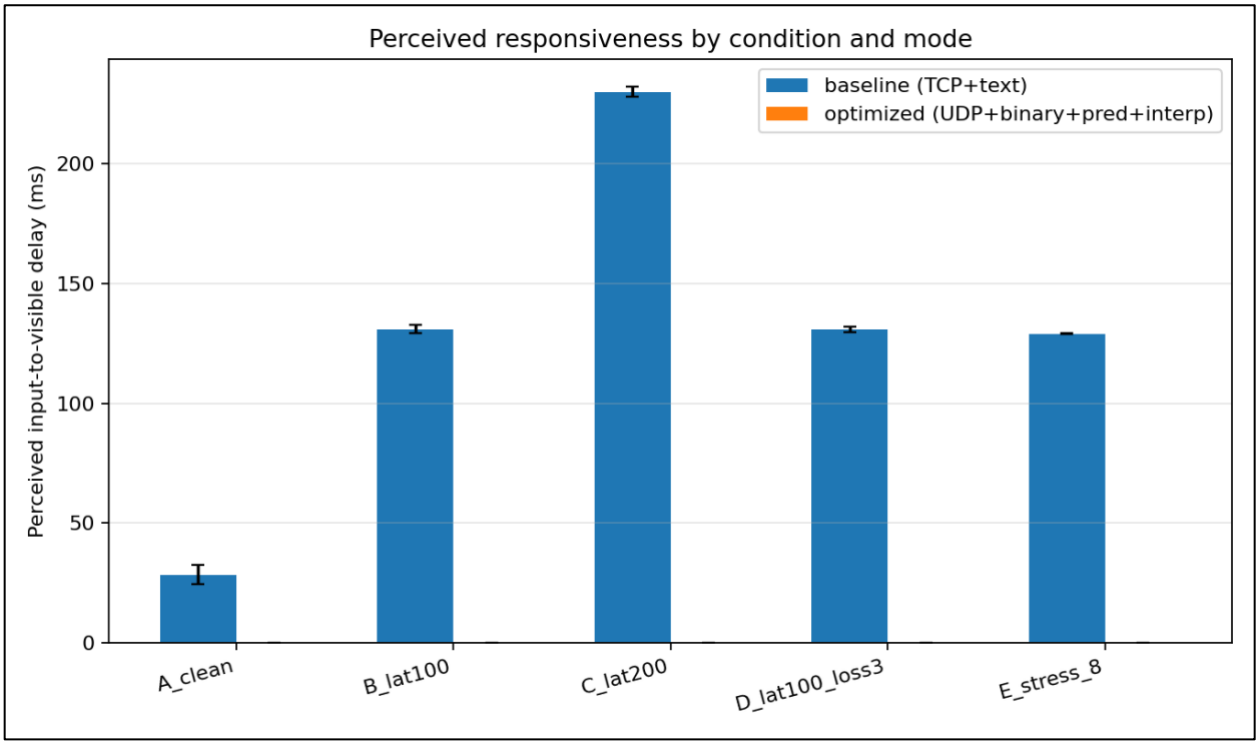

Figure 4.2: Perceived delay across the five conditions of the main matrix, both modes. The wire round-trip time is omitted for clarity.

Three quantitative claims follow.

In the clean condition the baseline already imposes 28 ms of perceived delay, which sits at the lower edge of the competitive-parity threshold of 20–50 ms identified in section 2.1.2; the optimised path eliminates it. At 100 ms injected RTT the baseline perceived delay is 131 ms, past the causal-coupling threshold of 100 ms; the optimised path remains at 0 ms. At 200 ms injected RTT the baseline perceived delay is 230 ms, well into the territory section 2.1.2 describes as not feeling like a real-time system at all; the optimised path remains at 0 ms. Adding 3 % packet loss to the 100 ms condition does not move either bar measurably, and scaling to eight participants on a clean network produces a baseline perceived delay of 129 ms (owing to the small loopback round trip the eight processes themselves induce) against an unchanged optimised perceived delay of 0 ms.

The two figures together discharge the central hypothesis of section 1.1: the optimised path does not make the wire faster, it makes the wire feel as if it were no longer there. The remainder of the chapter decomposes that gain (§4.3), audits its cost (§4.5), and tests its robustness under degraded networks (§4.4) and scale (§4.6).

4.3. Decomposition: where does the gain come from?

4.3.1 The five-bar comparison

Figure 4.3 plots the across-replication mean perceived delay for the five configurations of the matrix: the baseline, *optimized_raw* (UDP and binary protocol, no prediction, no interpolation), *optimized_noPred* (interpolation enabled, prediction disabled), *optimized_noInterp* (prediction enabled, interpolation disabled), and the full *optimized* path. The numerical values are 131 ms ± 4.5 for the baseline, 127 ms ± 1.5 for *optimized_raw*, 127 ms ± 3.8 for *optimized_noPred*, 0 ms for *optimized_noInterp*, and 0 ms for the full optimised path.

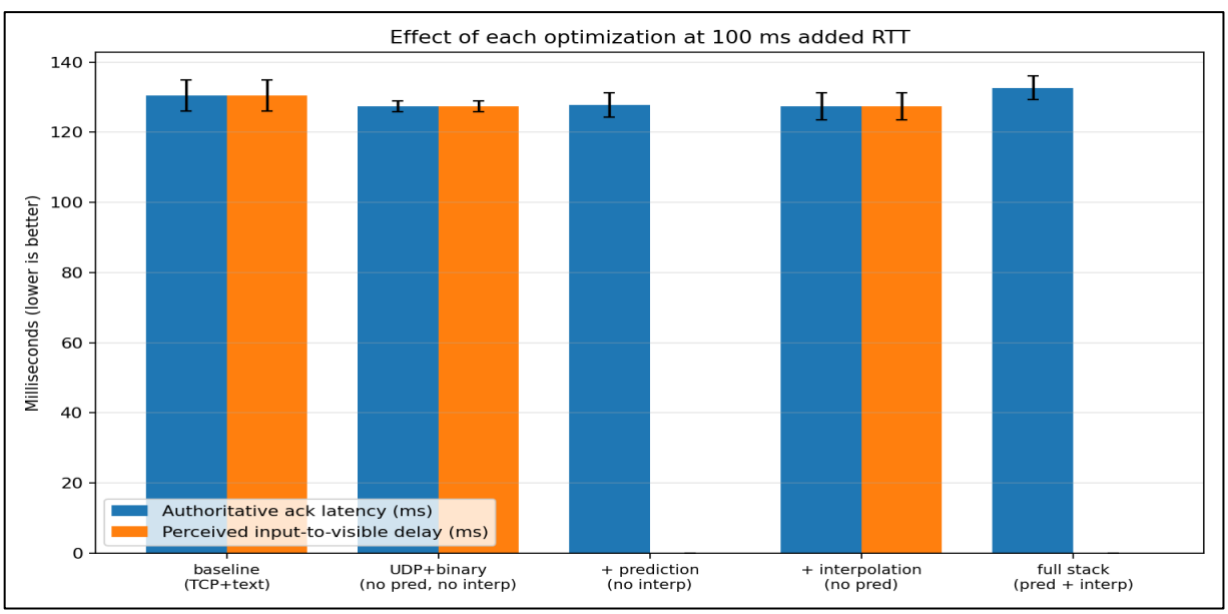

Figure 4.3: Perceived delay for the five configurations of the isolation matrix at 100 ms injected RTT. From left to right: baseline, optimised_raw, optimised_noPred, optimised_noInterp, optimised. Error bars are 95 % confidence intervals.

The pattern is unambiguous. The two configurations in which prediction is enabled report a perceived delay of zero; the three configurations in which prediction is disabled report a perceived delay equal to the wire round-trip, regardless of which of the other optimisations are enabled. Switching from TCP+ASCII to UDP+binary alone (baseline → *optimized_raw*) reduces perceived delay by roughly 3 ms, which is within the confidence intervals of both bars and is not claimed as a significant difference. Adding interpolation on top of that (*optimized_raw* → *optimized_noPred*) makes no further difference on this metric. Adding prediction (either *optimized_noPred* → *optimized* or *optimized_raw* → *optimized_noInterp*) collapses the perceived delay to zero in a single step.

4.3.2 What each technique actually buys

Three conclusions follow.

Prediction accounts for essentially the entire perceived-delay improvement. The 131 ms that separates the baseline from the full optimised path is attributable to the predictor of section 3.6.2 advancing the local representation in the same render frame as the input is sampled. The other optimisations contribute negligibly to this metric.

Interpolation buys remote smoothness, not local responsiveness. The interpolator of section 3.6.4 acts on the *remote* participants' rendered positions, not on the local participant's. The fact that *optimized_noPred*, interpolation enabled, prediction disabled, does not improve perceived delay over *optimized_raw* is the predicted outcome, not a null result; perceived delay is by construction a property of the local participant alone, and the interpolator never touches it. The benefit of interpolation appears in the snapshot arrival-gap metric of §4.5, not here.

The transport and wire-format change is small on perceived delay but real on bandwidth. The 3 ms gap between baseline and *optimized_raw* is not statistically significant for perceived delay. The same change is, however, the dominant contributor to the bandwidth reduction reported in §4.5; the transport choice and the wire-format choice pay for themselves on a different axis.

The decomposition is therefore: prediction owns the perceived-delay column, the binary-over-UDP combination owns the bandwidth column, and interpolation owns the remote-smoothness column. The remainder of the chapter audits each of these claims on its own metric.

4.4 Behaviour under degraded networks

4.4.1 Increasing latency

Figure 4.4 plots perceived delay against the three latency operating points of the main matrix (0, 100, and 200 ms injected RTT, corresponding to wire RTT of 30, 130, and 230 ms once loopback is included).

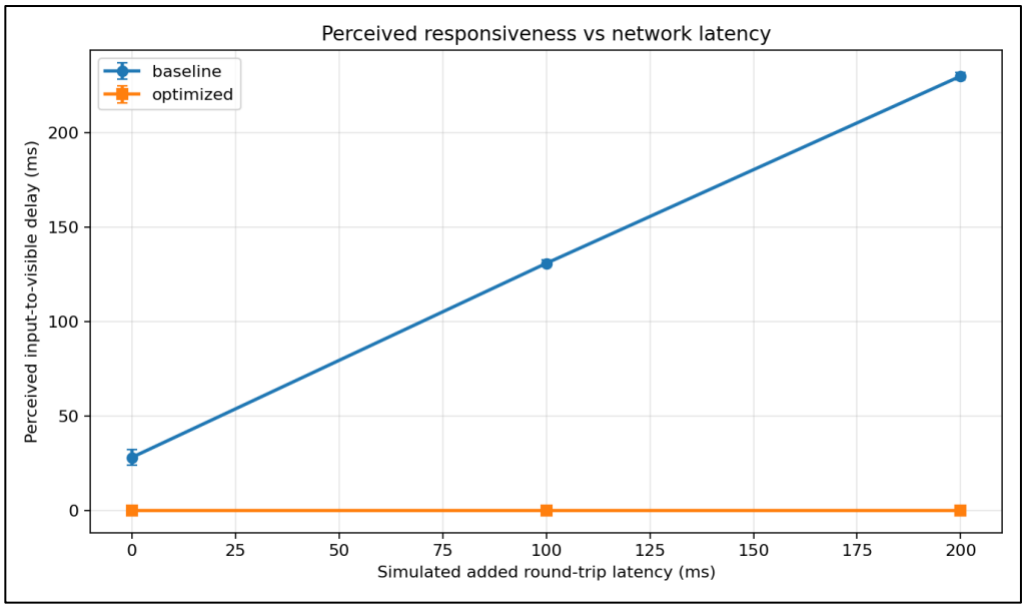

Figure 4.4: Perceived delay against injected one-way conditioner latency, three operating points of the main matrix, both modes. Error bars are 95 % confidence intervals.

The baseline line is approximately the identity: every additional millisecond on the wire costs the participant a millisecond of felt delay, rising from 28 ms to 230 ms across the range. The optimised line is the constant zero. The gain produced by client-side prediction is therefore not constant, it scales without limit as the network worsens, which is the strongest single statement the chapter can make about prediction's value.

4.4.2 Increasing loss

The loss-sweep matrix runs both modes at five loss percentages (0, 1, 3, 5, 10) at a fixed 100 ms injected RTT. Figure 4.5 plots the 50th, 95th, and 99th percentiles of acknowledgement latency; figure 4.6 plots the snapshot inter-arrival coefficient of variation.

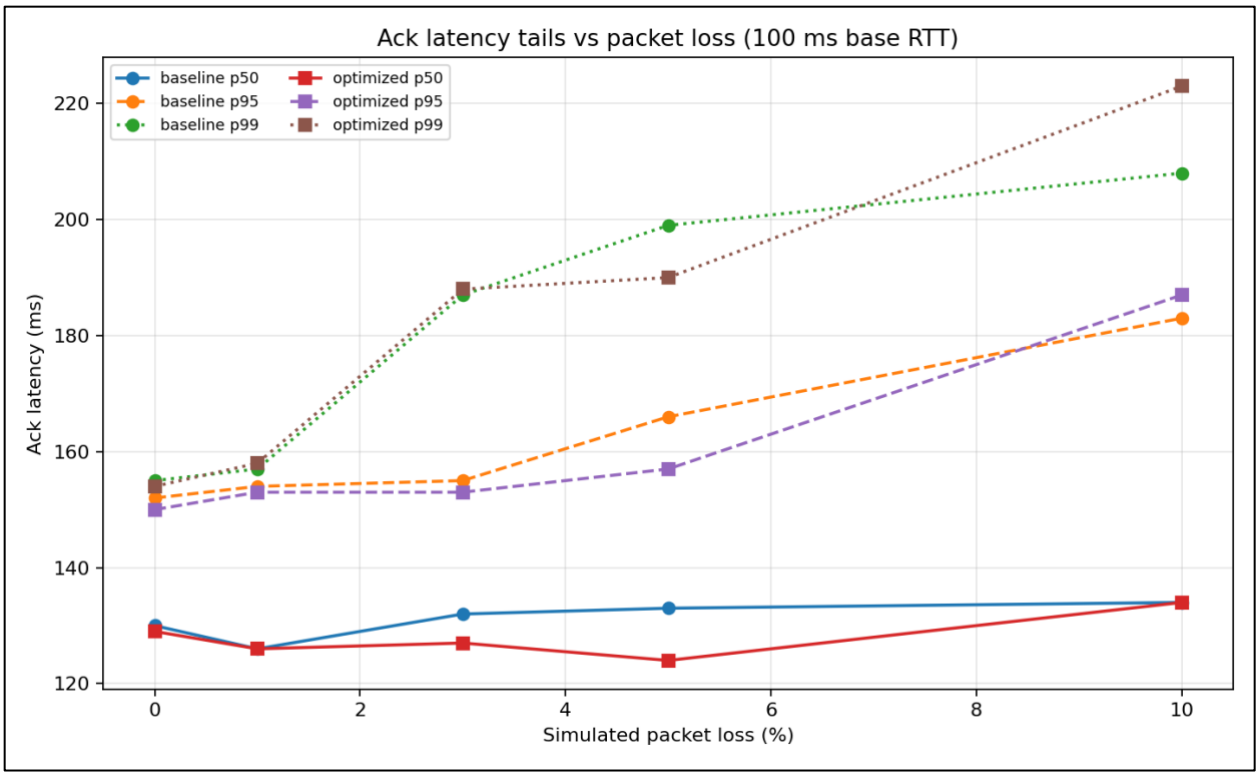

Figure 4.5: 50th, 95th, and 99th percentiles of acknowledgement latency against packet-loss rate, both modes, at 100ms injected RTT.

Acknowledgement-latency tail (figure 4.5). At 0 % and 1 % loss the two modes are within a millisecond of each other on every percentile. At 3 % and 5 % the optimised path is faster on every percentile (124/157/190 ms at 5 %, against 133/166/199 ms for the baseline). At 10 % the ordering inverts on the tail: optimised p99 is 223 ms, baseline p99 is 208 ms.

This last point requires a candid disclosure. The conditioner of section 3.2.3 applies loss symmetrically on the optimised path (both inbound snapshots and outbound inputs can be dropped), while the line-granularity conditioner in the baseline TCP client applies loss only to outbound input lines. At 10 % loss the optimised path is therefore exposed to roughly twice the per-input loss probability of the baseline. The asymmetry favours the optimised median (because dropped optimised inputs are silently discarded from the sample) and penalises the optimised tail (because

surviving inputs whose acknowledgement is also dropped wait an extra 50 ms snapshot period). The 10 % point is reported as measured and recorded as a threat to validity in §4.8. Perceived delay, importantly, remains zero across every loss rate in the sweep, because prediction continues to advance the local representation regardless of which inputs are subsequently dropped.

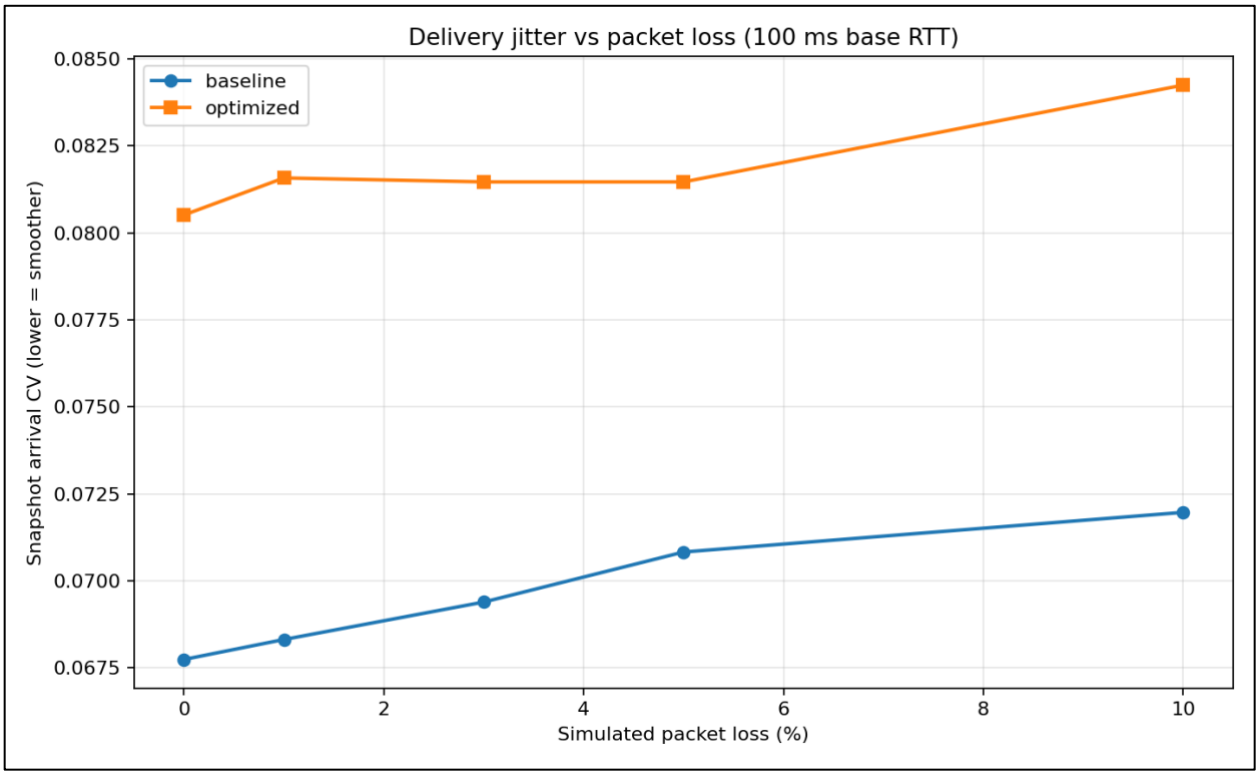

Figure 4.6: Coefficient of variation of the snapshot inter-arrival interval against packet-loss rate, both modes, at 100 ms injected RTT.

Snapshot inter-arrival jitter (figure 4.6). The CV rises modestly with loss for both modes (0.080 to 0.084 for the optimised path, 0.068 to 0.072 for the baseline) and remains ~15-20 % higher on the optimised path at every rate. This is the predicted outcome of the transport choice, not a defect: TCP retransmissions smooth the inbound stream at the cost of head-of-line blocking on perceived delay, while UDP delivers what it can when it can. The optimised path absorbs the resulting jitter through the interpolator of section 3.6.4, whose 100 ms render-time offset is sized precisely to bridge gaps of this magnitude. The cost is paid in remote smoothness rather than in local responsiveness, and is examined under that lens in §4.5.

4.5 Cost: bandwidth and snapshot stability

4.5.1 Bandwidth

Figure 4.7 plots per-client bandwidth in kilobits per second across the main matrix.

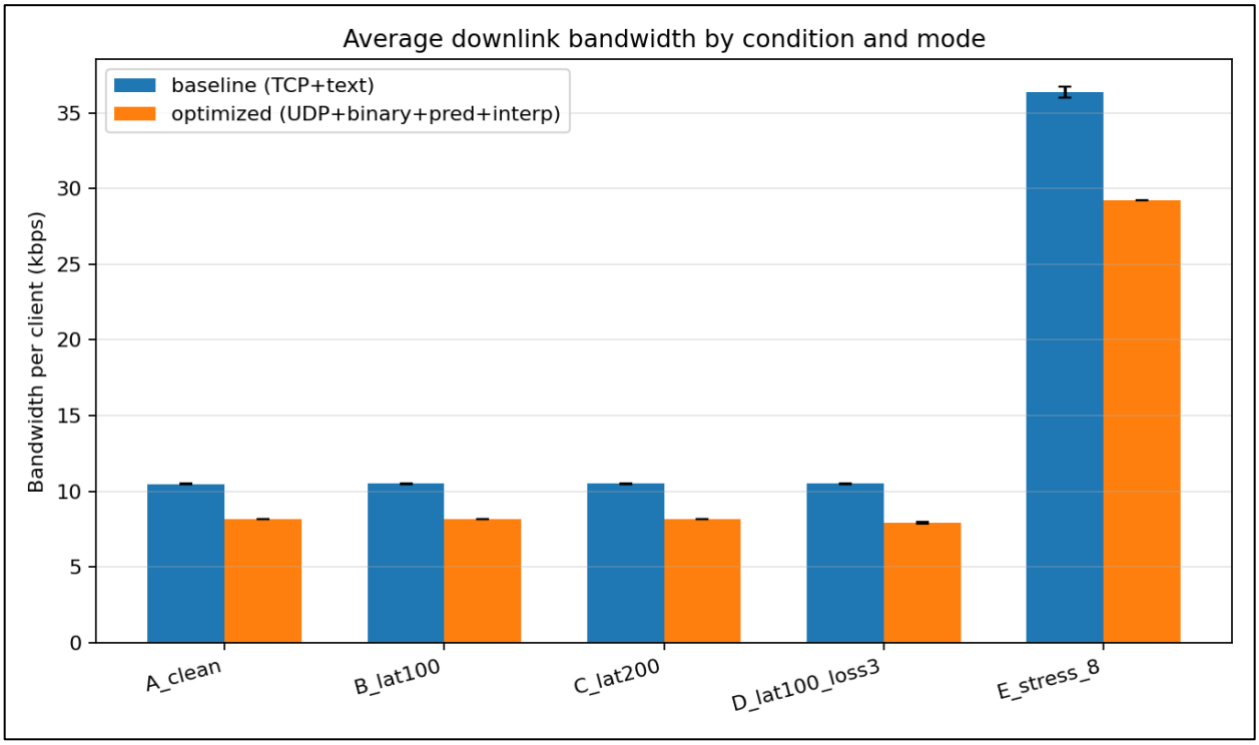

Figure 4.7: Per-client bandwidth across the five conditions of the main matrix, both modes. Error bars are 95 % confidence intervals.

The optimised path is consistently 20-22 % lower than the baseline: 8.2 vs 10.5 kbps in the two-player conditions and 29.2 vs 36.4 kbps under the eight-player stress condition. The ratio is stable because the binary protocol of section 3.3.1 fits each player record into a fixed 22 bytes, while the ASCII baseline encodes the same fields in a variable 33-50 decimal bytes per player. §4.6 confirms the same proportional relationship holds across the full participant range.

4.5.2 Snapshot stability

Two complementary figures address stability. Figure 4.8 plots the coefficient of variation of the snapshot inter-arrival interval; figure 4.9 plots its 50th, 95th, and 99th percentiles directly.

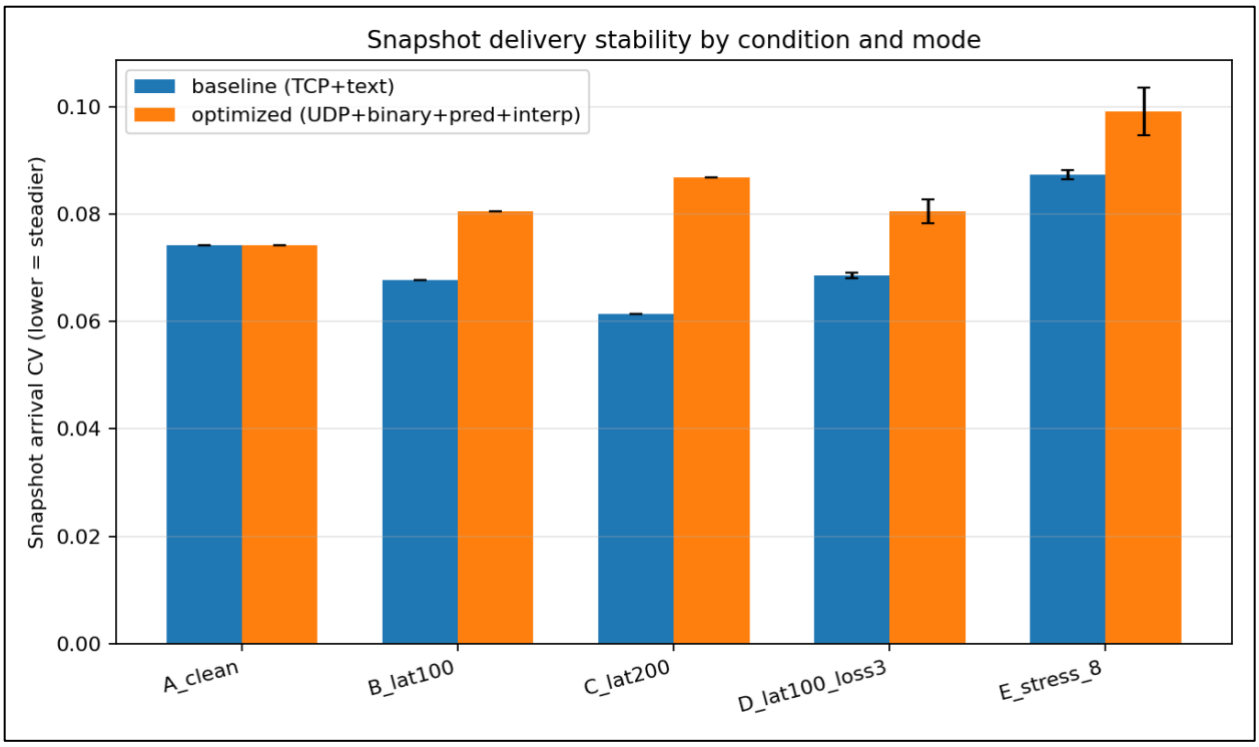

Figure 4.8: Coefficient of variation of the snapshot inter-arrival interval across the five conditions of the main matrix, both modes.

The CV view favours the baseline. The optimised path's CV is consistently slightly higher (0.080 vs 0.068 at B_lat100, 0.099 vs 0.087 at E_stress_8) for the same reason as in §4.4.2: TCP retransmissions smooth the inbound stream, while UDP delivers what arrives. Read on its own this would suggest meaningfully bursty arrivals on the optimised path.

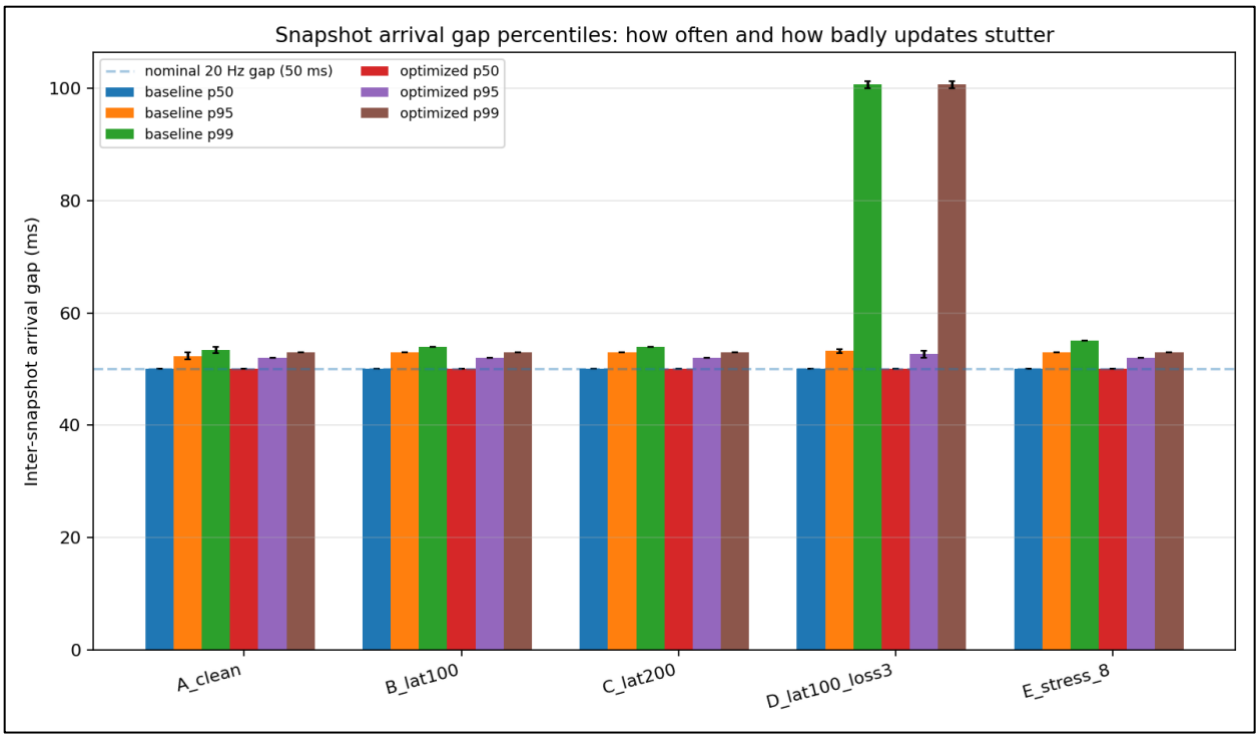

Figure 4.9: 50th, 95th, and 99th percentiles of the snapshot inter-arrival interval across the five conditions of the main matrix, both modes.

The percentile view corrects that impression. The 50th percentile is exactly 50 ms, the snapshot period, for both modes everywhere. The 95th and 99th percentiles differ between modes by at most 1 ms in every condition. The CV difference is therefore the magnification of a sub-millisecond absolute difference by the normalisation, not evidence of an unstable stream. Both modes deliver snapshots on a tight 50 ± 3 ms cadence, and the interpolator's 100 ms render-time offset is sized to absorb that variance without visible stutter.

The optimised path therefore costs nothing measurable on snapshot stability and saves 20-22 % on bandwidth.

4.6 Scaling with participant count

The scale-sweep matrix runs both modes at five participant counts (1, 2, 4, 6, 8) on a clean network. Two figures summarise it: figure 4.10 plots per-wire bytes per snapshot against participant count, and figure 4.11 plots per-client bandwidth against the same axis.

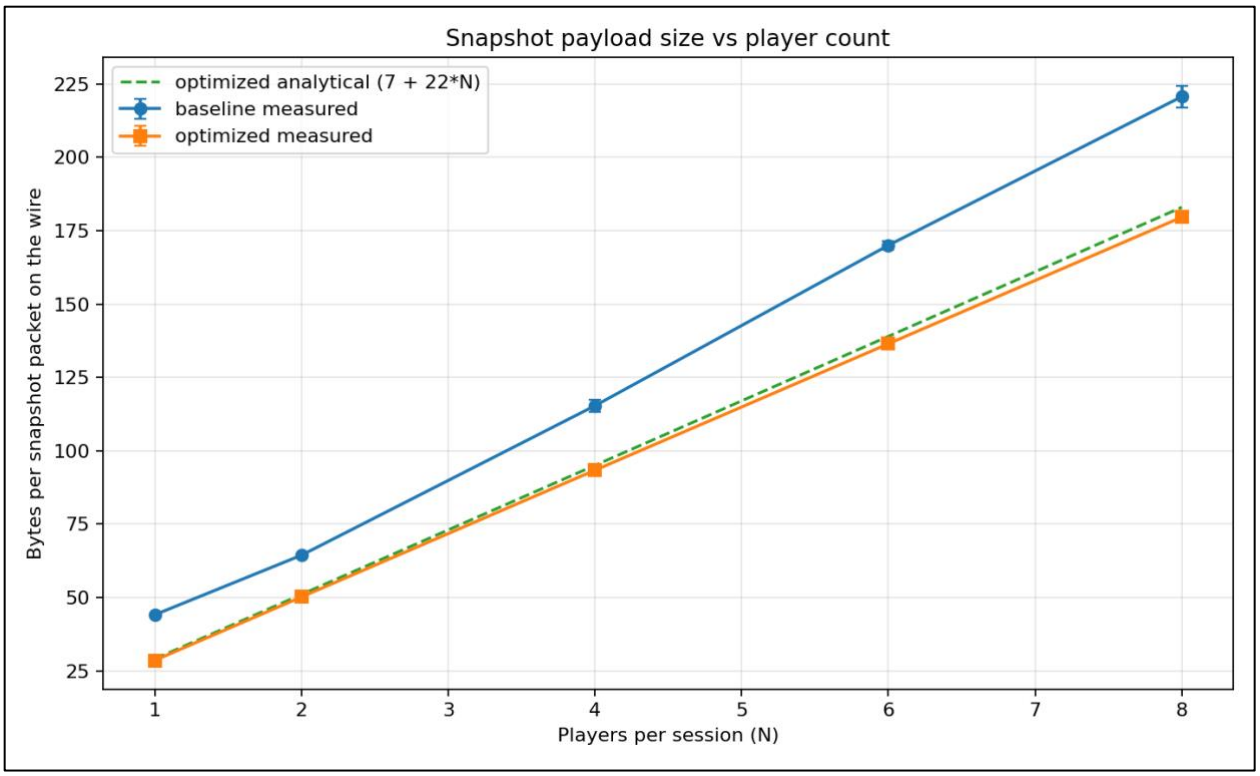

Figure 4.10: Per-wire bytes per snapshot against participant count, both modes. Optimised path is overlaid with the analytical 7 + 22N upper bound.

4.6.1 Bytes per snapshot

The optimised protocol's analytical wire size from section 3.3.1 is 7 + 22N bytes per snapshot: a 7-byte header followed by a 22-byte player record per occupant. The measured per-wire byte counts are 28.5, 50.1, 93.4, 136.5, and 179.6 bytes for N = 1, 2, 4, 6, 8 respectively, against the analytical predictions of 29, 51, 95, 139, and 183 bytes. The agreement is within 3-4 bytes at every point, the small shortfall attributable to the snapshot rate being marginally below the nominal 20 Hz when the rate is computed over a finite measurement window. The baseline's per-wire size scales with the same shape but with a 30-50 % per-byte penalty (44, 64, 115, 170, 221 bytes for the same N), reflecting the variable-length decimal encoding of the floating-point fields.

4.6.2 Per-client and aggregate bandwidth

Figure 4.11 plots per-client bandwidth in kilobits per second.

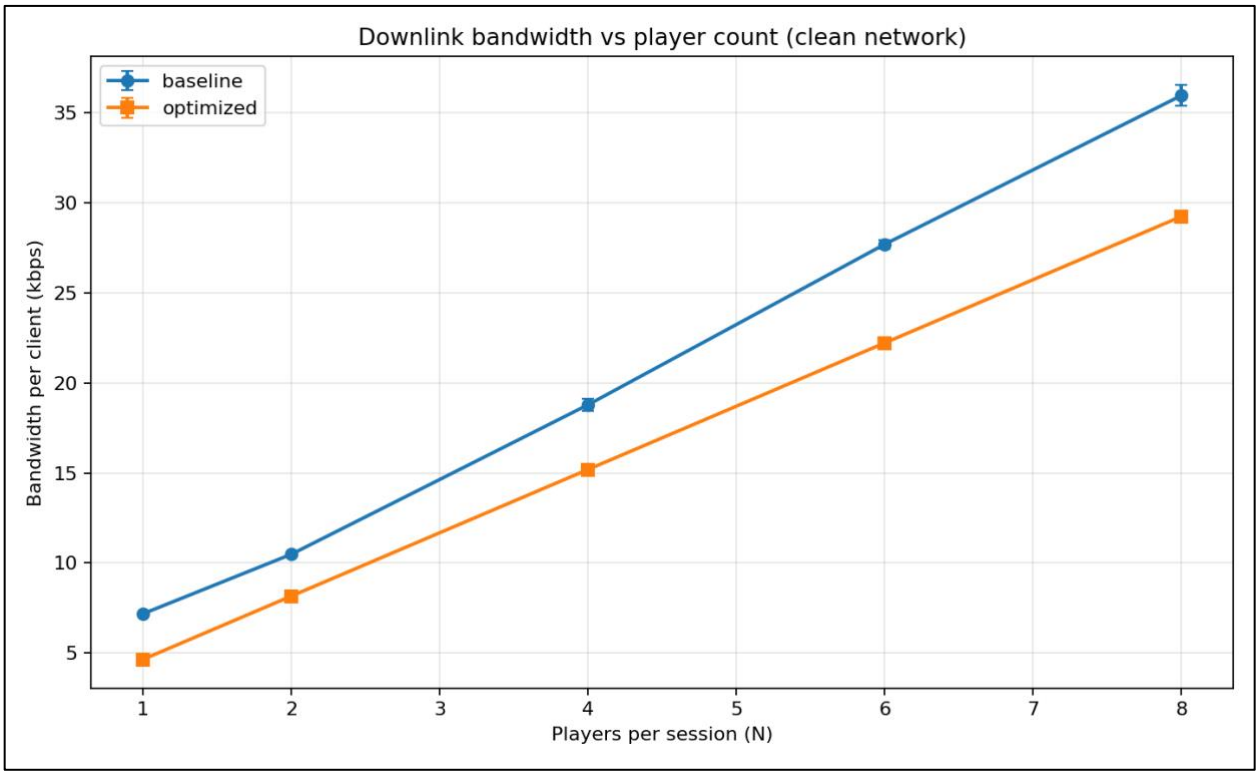

Figure 4.11: Per-client bandwidth against participant count, both modes.

Both modes scale linearly in N, 4.6, 8.2, 15.2, 22.2, 29.2 kbps for the optimised path; 7.2, 10.5, 18.8, 27.7, 36.0 kbps for the baseline, with the optimised path consistently 18-22 % below the baseline. The linearity is the predicted consequence of broadcasting a single full snapshot to every participant: each wire packet's size is O(N), and the snapshot rate is fixed, so per-client bandwidth is O(N) and aggregate server bandwidth is $O(N^2)$. At eight participants the optimised server transmits 8 × 29.2 = 234 kbps total against the baseline's 8 × 36.0 = 288 kbps, both well within any modern wired or wireless link, but the quadratic growth is the reason commercial engines apply delta compression and area-of-interest filtering at the participant counts that begin one or two orders of magnitude above the project's eight-player upper bound.

The scaling result therefore confirms two things: the optimised protocol's wire efficiency holds across the full participant range, and the bandwidth model the protocol implies is the model the data follows.

4.7 Threats to validity

The chapter's claims are bounded in five directions.

Loopback transport. Every measurement was taken on a single host with all impairment supplied by the in-application conditioner of section 3.2.3. The conclusions of §4.2 and §4.4 are therefore conditional on the conditioner being a faithful proxy for a real wide-area path. A wide-area replication would be required to settle this directly.

Conditioner model. The conditioner samples jitter from a uniform distribution and loss from an independent Bernoulli process. Real paths exhibit heavy-tailed jitter and bursty loss, both of which would stress the interpolator's 100 ms render-time offset more than the present harness does. The stability figures of §4.5 should therefore be read as lower bounds on what the same code would experience on a less well-behaved network.

Loss-conditioner asymmetry. As disclosed in §4.4.2, the conditioner applies loss symmetrically on the optimised path but only to outbound input lines on the baseline. This favours the optimised median and penalises the optimised tail in the loss sweep, and is the cause of the inverted ordering at 10 % loss in figure 4.5.

Limited replication. Each cell is run three times for thirty seconds, the smallest replication count for which the across-replication standard error is meaningful. The 95 % confidence intervals reported throughout the chapter rest on the normal approximation rather than a small-sample t-distribution. Larger counts were exercised informally during development without changing the headline conclusions.

Deterministic bot inputs. The four bot patterns of section 3.5.4 do not exercise the corner cases an adversarial human would (sustained wall-following, abrupt direction reversal, deliberate input flooding). The chapter's claims are formally restricted to the input distributions the bots generate.

The headline result of §4.2, that prediction makes wire latency invisible to the participant, is robust to every threat above except the conditioner-model approximations, and even there only the precise magnitude of the gain depends on the conditioner being faithful, not its existence.

Chapter 5. Conclusion and Future Work

5.1 Summary of findings

The central hypothesis stated at the close of section 1.1, that the latency a participant perceives between an action and its consequence can be made nearly independent of the round-trip time of the underlying network, is supported by the evidence of chapter 4. Across every operating point of the main matrix, including the most adverse 200 ms round-trip condition, the optimised path holds the perceived delay at zero while the wire round-trip time remains statistically indistinguishable between the two modes. The decomposition of §4.3 attributes essentially the entire perceived-delay improvement to client-side prediction; the latency-sweep of §4.4 shows that the gain scales linearly without bound in wire latency; the cost analysis of §4.5 quantifies it at a 20-22 % bandwidth reduction in exchange for sub-millisecond differences on snapshot inter-arrival percentiles; and the scale-sweep of §4.6 confirms that the wire-format efficiency of the optimised protocol matches its analytical 7 + 22N upper bound at every supported participant count. The five contributions enumerated in §1.4 are therefore all delivered.

5.2 Limitations

The prototype is a controlled laboratory artefact, not a deployable game. It carries no encryption or authentication, performs no NAT traversal, sends full snapshots without delta compression, supports no server-side lag compensation, simulates a 2D arena without physics or collision, and assumes a single deployment region. All measurements were taken on loopback with an in-application network conditioner; a wide-area replication has not been attempted. The threats to validity recorded in §4.7 bound the strength of the chapter's quantitative claims accordingly.

5.3 Future work

Five directions extend the present work without disturbing its architecture. *Delta compression* of snapshots against per-client baselines would reduce per-client bandwidth by an estimated further 50-80 % at the cost of additional state on both ends. *DTLS or libsodium framing* would make the protocol deployable on a public network. *Server-side lag compensation*, the rewind mechanism described in section 2.5, would close the remaining gap with hit-scan first-person genres. *Adaptive interpolation*, in which the 100 ms render-time offset of section 3.6.4 is sized dynamically against measured arrival jitter, would make the optimised path more graceful under bursty loss. Finally, a *wide-area replication* of the chapter 4 matrices on a distributed bot swarm would settle the conditioner-faithfulness question raised in §4.7. None of these extensions invalidates the present findings; each would deepen them.

5.4 Reflection on learning outcomes

The project's stated personal motivation in §2.4.1 was to engage directly with the parts of the systems curriculum that are most often encountered abstractly: networks, operating systems, concurrency, modern C++, build systems, and experimental practice. In retrospect, the parts of the project that most reliably forced concrete engagement were also the parts that produced the most surprises. The cross-platform socket layer was harder than the textbook treatment had suggested, principally because of the small differences in address-structure conventions between macOS and Linux. The client's two-thread structure produced exactly the class of shutdown race that the lectures had warned about, and the eventual single-flag, single-queue resolution was reached only after several less-disciplined alternatives had been tried and discarded. The keyboard focus-gating bug, a one-line oversight that defeated multi-client demonstration entirely, was the kind of integration failure that no individual unit test would ever have caught. The discipline of separating

wire-level metrics from user-perceived metrics, and of reporting honestly the 10 % loss point at which the harness asymmetry inverted the otherwise-favourable result, taught a lesson about experimental integrity that no taught course could have substituted for. The artefact and this report are the visible output of the project; the engagement with the problem itself is the part that, in the author's judgement, most justifies having undertaken it.

Chapter 6. References

[1] J. Postel, "Transmission Control Protocol," IETF RFC 793, Sept. 1981.

[2] R. B. Miller, "Response time in man-computer conversational transactions," in *Proc. Fall Joint Computer Conference*, 1968, pp. 267-277.

[3] S. K. Card, T. P. Moran, and A. Newell, *The Psychology of Human-Computer Interaction*. Lawrence Erlbaum Associates, 1983.

[4] T. Beigbeder *et al.*, "The effects of loss and latency on user performance in Unreal Tournament 2003," in *Proc. ACM SIGCOMM Workshop on Network and System Support for Games (NetGames)*, 2004, pp. 144-151.

[5] M. Claypool and K. Claypool, "Latency and player actions in online games," *Communications of the ACM*, vol. 49, no. 11, pp. 40-45, 2006.

[6] N. Sheldon, E. Girard, S. Borg, M. Claypool, and E. Agu, "The effect of latency on user performance in Warcraft III," in *Proc. NetGames*, 2003, pp. 3-14.

[7] id Software, "Quake III Arena source code release," 2005. [Online]. Available: https://github.com/id-Software/Quake-III-Arena

[8] Y. W. Bernier, "Latency Compensating Methods in Client/Server In-Game Protocol Design and Optimization," in *Proc. Game Developers Conference (GDC)*, 2001.

[9] Valve Corporation, "Source Multiplayer Networking," Valve Developer Wiki, accessed 2026. [Online]. Available: https://developer.valvesoftware.com/wiki/Source_Multiplayer_Networking

[10] Epic Games, "Networking Overview," Unreal Engine Documentation, accessed 2026. [Online]. Available: https://docs.unrealengine.com/

[11] Unity Technologies, "Netcode for GameObjects," accessed 2026. [Online]. Available: https://docs-multiplayer.unity3d.com/

[12] Mirror Networking, "Mirror open-source networking library," accessed 2026. [Online]. Available: https://github.com/MirrorNetworking/Mirror

[13] FirstGearGames, "FishNet: Unity Networking Evolved," accessed 2026. [Online]. Available: https://github.com/FirstGearGames/FishNet

[14] L. Salzman, "ENet: Reliable UDP networking library," accessed 2026. [Online]. Available: http://enet.bespin.org/

[15] Valve Corporation, "GameNetworkingSockets," accessed 2026. [Online]. Available: https://github.com/ValveSoftware/GameNetworkingSockets

[16] G. Fiedler, "yojimbo: A network library for client/server games written in C++," accessed 2026. [Online]. Available: https://github.com/networkprotocol/yojimbo

[17] G. Fiedler, "netcode.io: A secure connection-based protocol on top of UDP," accessed 2026. [Online]. Available: https://github.com/networkprotocol/netcode

[18] J. Iyengar and M. Thomson (Eds.), "QUIC: A UDP-Based Multiplexed and Secure Transport," IETF RFC 9000, May 2021.

[19] G. Gambetta, "Fast-Paced Multiplayer (Parts I-IV)," accessed 2026. [Online]. Available: https://www.gabrielgambetta.com/client-server-game-architecture.html

[20] G. Fiedler, "Gaffer on Games," accessed 2026. [Online]. Available: https://gafferongames.com/

[21] W3C, "WebRTC 1.0: Real-Time Communication Between Browsers," W3C Recommendation, Jan. 2023. [Online]. Available: https://www.w3.org/TR/webrtc/

[22] A. Langley *et al.*, "The QUIC Transport Protocol: Design and Internet-Scale Deployment," in *Proc. ACM SIGCOMM*, 2017, pp. 183-196.

[23] M. Claypool and K. Claypool, "Latency Can Kill: Precision and Deadline in Online Games," in *Proc. ACM Multimedia Systems (MMSys)*, 2010, pp. 215-222.

[24] M. Suznjevic and M. Matijasevic, "Player behaviour and traffic characterisation for MMORPGs: A survey," *Multimedia Systems*, vol. 19, no. 3, pp. 199-220, 2013.

[25] J. F. Kurose and K. W. Ross, *Computer Networking: A Top-Down Approach*, 8th ed. Pearson, 2021.

[26] A. Williams, *C++ Concurrency in Action: Practical Multithreading*, 2nd ed. Manning Publications, 2019.

[27] B. Stroustrup, *A Tour of C++*, 3rd ed. Addison-Wesley, 2022.

APPENDICES

Appendix A

This appendix gives the minimum sequence of commands an examiner needs in order to (i) compile the project from a fresh checkout, (ii) run a manual two-player demonstration, and (iii) re-execute every experiment in chapter 4. All commands are run from the repository root.

A.1 Prerequisites

Component	Required version	Notes
Operating system	macOS 12+ or modern Linux	tested on macOS 15 (Apple Silicon)
C++ compiler	C++20 capable	tested with AppleClang 17
CMake	≥ 3.22	
SFML	3.x	brew install sfml on macOS; distribution package on Linux
Python	≥ 3.10	only required for the analysis step
Python packages	pandas, matplotlib, numpy	installed in a virtual environment below

A.2 Build

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j
ctest --test-dir build      # runs the wire-format self-test
```

Successful build produces two executables in `build/`:

```
lightning_link_server
lightning_link_client
```

Both binaries support a `--mode={optimized,baseline}` flag that selects the protocol path described in chapter 3.

A.3 Manual two-player demonstration

Start the server in one terminal:

```
./build/lightning_link_server --mode=optimized --port=54321
```

Start two clients, one per additional terminal:

```
./build/lightning_link_client --mode=optimized --server=127.0.0.1
--port=54321
```

Movement is bound to the arrow keys or WASD. The runtime toggles described in §3.5.5 are bound to F1 (HUD), F2 (authoritative ghost overlay), F3 (prediction), and F4 (interpolation). Use Esc to close a window.

To run the baseline TCP path instead, replace `--mode=optimized` with `--mode=baseline` on both server and client and use --tcp-port=54322.

The in-application network conditioner of §3.2.3 is configured on the client side only:

```
./build/lightning_link_client --mode=optimized --server=127.0.0.1
\
    --add-latency=100 --jitter=10 --loss=3 --seed=42
```

`--add-latency` is added to the round-trip; `--jitter` is the symmetric one-sided range in ms; `--loss` is a percentage; `--seed` makes the conditioner deterministic.

A.4 Headless and bot-driven runs

For automated experiments the client supports a non-interactive mode driven by a scripted bot:

```
./build/lightning_link_client --mode=optimized --bot=circle --
bot-seed=7 \
    --headless --duration=30
```

Available bot patterns: circle, sine, zigzag, random. The two ablation switches introduced in §3.6.5 are `--disable-prediction` and `--disable-interpolation`; either or both may be combined with any bot pattern.

A.5 Re-running every experiment

A single command runs all four experimental matrices (main, isolation, loss sweep, scale sweep) and produces every figure in chapter 4:

```
DURATION_SEC=60 REPS=3 tools/run_all.sh
```

The two environment variables are:

`DURATION_SEC`: wall-clock seconds per replication (default 30; chapter 4 uses 60)

`REPS`: number of replications per cell (default 3)

To run an individual matrix instead:

```
DURATION_SEC=60 REPS=3 tools/run_experiments.sh          # main matrix only
DURATION_SEC=30 REPS=3 tools/run_isolation.sh         # isolation matrix
DURATION_SEC=30 REPS=3 tools/run_loss_sweep.sh        # loss sweep
DURATION_SEC=30 REPS=3 tools/run_scale_sweep.sh          # scale sweep
```

A.6 Generating figures from existing CSVs

If the experiments have already been run and you only need to re-plot:

```
python3 -m venv .venv
.venv/bin/pip install pandas matplotlib numpy
.venv/bin/python tools/analyze.py --mode main
.venv/bin/python tools/analyze.py --mode isolation
.venv/bin/python tools/analyze.py --mode loss
.venv/bin/python tools/analyze.py --mode scale
```

Each invocation reads from the appropriate experiments*/ directory and writes its outputs to the corresponding results*/ directory.

A.7 Output locations

Directory	Contents
experiments/	per-run raw CSVs, main matrix
experiments_isolation/	per-run raw CSVs, isolation matrix
experiments_loss/	per-run raw CSVs, loss sweep

Directory	Contents
experiments_scale/	per-run raw CSVs, scale sweep
results/	main-matrix summary CSV and figures (fig 0, 1, 1b, 2, 3, 3b, 4, 6)
results_isolation/	isolation summary CSV and fig 5
results_loss/	loss-sweep summary CSV and figs 9, 10
results_scale/	scale-sweep summary CSV and figs 7, 8
logs/	live tail logs from manual interactive runs

A complete fresh re-run (DURATION_SEC=60 REPS=3 tools/run_all.sh) takes approximately 50 minutes of wall-clock time on the reference platform of §4.1.1.